SILVA: A Scalable Incremental Layered Sparse Value-Flow Analysis

JIAYI WANG and YU WANG, State Key Laboratory for Novel Software Technology, School of Computer Science, Nanjing University, Nanjing, China
KE WANG, Visa Research, Foster City, California, USA
LINZHANG WANG, State Key Laboratory for Novel Software Technology, School of Computer Science, Nanjing University, Nanjing, China

Layered sparse value-flow analysis (SVFA) is a prominent static analysis for resolving program dependencies. Despite the significant progress, SVFA still suffers from scalability issue. In light of the natural, continuous evolution of software, we introduce SILVA, the first incremental layered SVFA that scales to large, real-world programs efficiently. At the core of SILVA lies a novel incremental pointer analysis and incremental Mod-Ref analysis. Our extensive experiments on large-scale, real-world C/C++ programs demonstrate its effectiveness: SILVA achieves nearly a 7× speedup over SVF, the state-of-the-art layered SVFA, without losing any precision. Moreover, our incremental pointer and Mod-Ref analysis algorithms are 12× and 5× faster than existing methods, respectively. Regarding the impact of the size of the code changes on SILVA's effectiveness, we find that SILVA outperforms SVF for changes up to 10K lines—well beyond the typical scope of code commits in real-world software development.

CCS Concepts: • **Software and its engineering → Automated static analysis**; • **Theory of computation → Program analysis**;

Additional Key Words and Phrases: value-flow analysis, incremental analysis, pointer analysis

ACM Reference format:
Jiayi Wang, Yu Wang, Ke Wang, and Linzhang Wang. 2025. SILVA: A Scalable Incremental Layered Sparse Value-Flow Analysis. *ACM Trans. Softw. Eng. Methodol.* 34, 8, Article 229 (October 2025), 40 pages.
https://doi.org/10.1145/3725214

1 Introduction

Value-flow analysis (VFA), which resolves both data and control dependences of a program, has obtained significant success across diverse application domains. Originally adopted in software debugging [70, 71] and compiler optimization [14, 55], VFA has more recently been shown highly effective in uncovering various types of bugs. These include memory leaks [73], double free [13],

This work was partially supported by the National Natural Science Foundation of China under Grant Nos. 62032010, 62232001, and 62202220, Collaborative Innovation Center of Novel Software Technology and Industrialization, and the Fundamental Research Funds for the Central Universities No. 2024300364.
Authors' Contact Information: Jiayi Wang, State Key Laboratory for Novel Software Technology, School of Computer Science, Nanjing University, Nanjing, China; e-mail: jiayiwang@smail.nju.edu.cn; Yu Wang (corresponding author), State Key Laboratory for Novel Software Technology, School of Computer Science, Nanjing University, Nanjing, China; e-mail: yuwang_cs@nju.edu.cn; Ke Wang (corresponding author), Visa Research, Foster City, California, USA; e-mail: kewang@visa.com; Linzhang Wang, State Key Laboratory for Novel Software Technology, School of Computer Science, Nanjing University, Nanjing, China; e-mail: lzwang@nju.edu.cn.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7392/2025/10-ART229
https://doi.org/10.1145/3725214

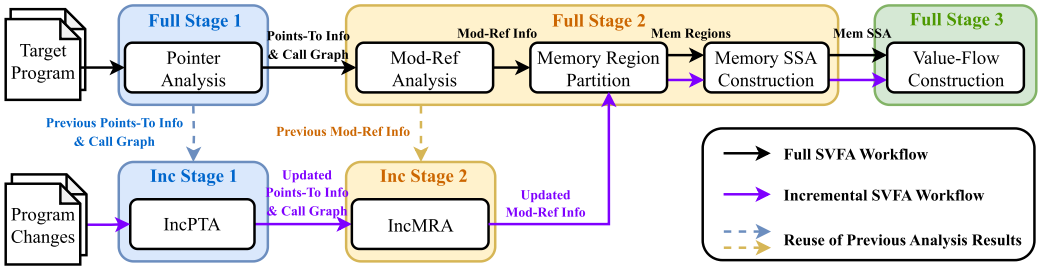


Fig. 1. Workflows of SVFA (connected by ⟶) and SILVA (connected by ⟶).

null pointer dereference [3], and use-after-free [75]. Unfortunately, despite the rapid advancement, VFA still faces critical challenges to be effective at industrial scale [44, 52, 63].

Later, **sparse value-flow analysis (SVFA)** [7, 51, 52, 60, 69] has been proposed to address the scalability issue. Among various SVFA techniques, the "layered" approach [7, 60] is one of the most successful. The key idea of layered SVFA is to first employ a pre-analysis to over-approximate the program dependencies, which are then refined via the flow of values on **sparse value-flow graphs (SVFGs)**. A prime example of layered SVFA is SVF [60], which is currently a highly popular analysis with a broad range of downstream applications such as program repair [74], static alert ranking [35], software defect detection [5, 6, 30, 39, 75], and code representation learning [58]. On the other hand, as pointed out by prior work [4, 52], layered SVFA still has difficulties in scaling to large, real-world programs. For instance, our preliminary study shows that SVF takes nearly 5 hours to analyze PostgreSQL, a well-known relational database system that has nearly 1 million lines of code. Moreover, software is constantly evolving as developers make code commits on a regular basis [21, 82], which makes it very expensive to perform layered SVFA on the entire program after every code commit. In light of this, we take the perspective of incremental analysis to further improve the performance and efficiency of layered SVFA. The idea is to analyze only the code that has changed or has been affected by the change. Compared to the whole program, the changes and their impact usually span much smaller code. Thanks to the reduced complexity, the analysis can be made much more efficient. In fact, existing studies have plenty of success in utilizing incremental approaches to accelerate the original static analysis algorithms [2, 33, 80].

To adopt an incremental approach for layered SVFA, the first challenge we must resolve is to identify the modules within its analysis framework that cause the inefficiency. Figure 1 illustrates the three stages in which layered SVFA techniques are carried out. In Stage 1, a **pointer analysis (PTA)** is applied to the program to discover the pointer information. Subsequently, in Stage 2, SVFA performs dataflow analyses based on the discovered pointer information. Specifically, a "**Mod-Ref analysis (MRA)**" is first conducted to identify the set of memory locations indirectly modified/referenced (Mod/Ref) by each function. Then, SVFA partitions the memory locations into a set of regions based on the Mod/Ref set and inserts the annotations for indirect accesses to memory regions into the analyzed code, effectively expanding the original SSA into Memory SSA. Finally, in Stage 3, the value-flows in the program are captured by the "Value-Flow Construction" module which produces a SVFG. Among all the analytical modules in SVFA, we identify that PTA and MRA, the two fundamental pre-analysis modules with non-linear time complexity, are the performance bottlenecks. Specifically, we observe that on several real-world programs (e.g., Git, PostgreSQL), SVF spends nearly 20% and 70% of its total analysis time on PTA and MRA, respectively, while the subsequent modules with linear time complexity (i.e., memory region partition, MSSA construction, and value-flow construction) account for less than 10% of the total analysis time. In response, we design incremental algorithms for PTA and MRA.

```
1    int  *a = malloc(); // a points to o1
2    int **x = malloc(); // x points to o2
3    y = x; // y points to o2
4  - *x = a; // o2 points to o1
5    *y = a; // o2 points to o1
```

Fig. 2. A deletion scenario for which the existing algorithm produces incorrect results.

Although significant progress has been made, we found two severe limitations of the state-of-the-art incremental PTA (i.e., IPA [33]). First, we discover that IPA produces incorrect results under certain circumstances. As illustrated in Figure 2, given the constraints from statements at lines 1 to 3, we have a pointing to $o1$, and x and y pointing to the same object $o2$, with the store statements at lines 4 and 5 causing $o2$ to point to $o1$. When the statement at line 4 is deleted, IPA would incorrectly conclude that $o2$ no longer points to $o1$. However, since the store at line 5 remains, $o2$ still points to $o1$. This error arises from ignoring the association between the deleted statement and other statements. To address this, we design rules to associate different pointer statements, thereby enabling the correct handling of the aforementioned scenarios. Taking Figure 2 as an example, during analysis, our rules can bind the points-to constraint (i.e., $o2$ points to $o1$) with the two stores that derive this constraint (i.e., $*x = a$ and $*y = a$). When a deletion occurs, we first maintain the binding relationship and then decide whether to remove the points-to constraint based on this binding relationship. Our second focus lies in optimizing the IPA algorithm to enhance efficiency. Our insight is to minimize redundant computation by deferring the incremental update of a pointer until all changes affecting the pointer have been gathered. For example, when changes propagate to a pointer through a constraint path, IPA immediately updates the pointer and propagates the subsequent changes. However, this eager approach has its limitations: If a pointer is affected by multiple changes along different paths, "eagerly" processing it can become frequent but redundant. In contrast, we accumulate all changes that propagate to a particular pointer and only update the pointer once by consolidating these changes, while also reducing the computation overhead introduced by propagating the subsequent changes after updating the pointer. Our proposed incremental PTA algorithm is flow- and context-insensitive, matching the precision of the most commonly used full PTA in SVFA. Moreover, we have demonstrated that our algorithm can produce analysis results that are consistent with the full analysis.

The other bottleneck module, MRA, is an instance of **function-summary-based dataflow analysis (FSA)** [24]. To incrementally update function summaries, existing methods [2, 49, 56, 57, 66, 80] typically follow the classic reset-recompute approach [16]. However, these methods suffer from significant scalability issues due to their coarse-grained nature, as they propagate changes in a function's summary to all its callers, leading to redundant computations for callers whose summaries remain unchanged. To address this, we propose a fine-grained algorithm that propagates only the necessary elements. Moreover, we present an early-stop strategy during the function-summary propagation, which uses intra-procedural summaries to filter the propagated inter-procedural summary change elements, allowing the summary update process to stop as early as possible. We instantiate our FSA to create incremental MRA, which accepts the output of our incremental PTA, and incrementally updates the Mod/Ref summaries for the entire program.

We realize our two incremental algorithms for layered SVFA into a new tool named **Scalable Incremental Layered sparse Value-flow Analysis (SILVA)**. We evaluate SILVA on five open source projects on GitHub using their actual code commits. Compared with the state-of-the-art full layered SVFA tool SVF [60], SILVA is on average 7× faster when running on each new version of the program (ranging from hundreds of thousands to even millions of lines of code) while achieving the same accuracy of the original SVF. More specifically, SILVA achieves an average acceleration of 650× in PTA and 10× in MRA. This effectively alleviated the bottlenecks in layered SVFA, reducing what previously took several hours of full analysis to just a dozen minutes. Furthermore, we compare our incremental PTA and MRA against the state-of-the-art. Results show that our incremental PTA is over 12× faster than the state-of-the-art incremental PTA algorithm IPA [33], and our incremental

Table 1. Constraints for Andersen's PTA

Type	Statement	Constraint Edge	Points-To Constraint
Address	$p = \textbf{alloca}\ o$	$p \xleftarrow{\text{addr}} o$	$o \in pts(p)$
Copy	$p = q$	$p \xleftarrow{\text{copy}} q$	$pts(q) \subseteq pts(p)$
Phi	$p = \phi(p_1, p_2, \ldots)$	$\forall p_i : p \xleftarrow{\text{copy}} p_i$	$\forall p_i : pts(p_i) \subseteq pts(p)$
Call	$t = f(a_1, a_2, \ldots)$	$\forall \text{return } r \text{ in } f(b_1, b_2, \ldots) :$ $t \xleftarrow{\text{copy}} r$ $\forall i : b_i \xleftarrow{\text{copy}} a_i$	$\forall \text{return } r \text{ in } f(b_1, b_2, \ldots) :$ $pts(r) \subseteq pts(t)$ $\forall i : pts(a_i) \subseteq pts(b_i)$
Load	$p = *q$	$p \xleftarrow{\text{load}} q$	$\forall o \in pts(q) : pts(o) \subseteq pts(p)$
Store	$*p = q$	$p \xleftarrow{\text{store}} q$	$\forall o \in pts(p) : pts(p) \subseteq pts(o)$
Offset	$p = \&q.f$	$p \xleftarrow{f} q$	$\forall o \in pts(q) : o.f \in pts(p)$

FSA algorithm is more than 5× faster than the reset-recompute approach when performing MRA. Additionally, we explore the performance of SILVA under different code change scales. The results indicate that even with changes involving thousands of lines of code, SILVA always performs faster than a full analysis.

In summary, the contributions of this article are as follows:

- —We propose an incremental approach to accelerate the layered SVFA.
- —We propose two incremental algorithms, respectively for the two bottleneck modules of layered SVFA, PTA and MRA.
- —We implement our incremental PTA and MRA for SVFA, resulted in a new tool, called SILVA (https://github.com/Patrickstarrrrr/SILVA). Our extensive evaluation of SILVA on real-world, large-scale programs demonstrates the significant performance improvement of SILVA over full analysis and other incremental algorithms.

2 Background

In this section, we provide background knowledge on the two bottleneck modules of layered SVFA—PTA and MRA.

2.1 PTA

PTA is a technique in static program analysis used to determine the set of memory locations that a pointer variable can point to during the execution of a program. It is the cornerstone of the development and research of other program analysis techniques. In this article, we focus on the classic and widely used Andersen's analysis [1], which is the most precise among all flow- and context-insensitive pointer analyses [63].

Andersen's analysis uses a constraint-based approach to compute the points-to sets (Definition 2.1) for all pointer variables in a program, based on inclusion constraints derived from different program statements listed in Table 1.

Definition 2.1. The *points-to set* of a variable p, $pts(p)$, is a set of memory locations (objects) possibly pointed by p.

Andersen's analysis first traverses the statements in the program and generates different constraint edges based on the types of statements, thereby transforming the target program into a

constraint graph (Definition 2.2). Subsequently, points-to sets can be solved by computing the transitive closure of the constraint graph, ensuring that for every pair of nodes p, q in the graph, if there exists a CopyEdge $p \xleftarrow{\text{copy}} q$, then $pts(p)$ and $pts(q)$ are the minimum points-to sets ensuring $pts(q) \subseteq pts(p)$.

Definition 2.2. The *constraint graph* of a program is a directed labeled graph $\mathcal{G} = \{\mathcal{N}, \mathcal{E}, pts\}$. $\mathcal{N}$ is a set of nodes, each of which corresponds to a variable or memory object in the program; $\mathcal{E} : (\mathcal{N} \times \mathcal{T} \times \mathcal{N})$ is a set of labeled edges between nodes, where $\mathcal{T} = \{\text{addr, copy, load, store, offset}\}$ is the set of types of five kinds of constraint edges; and $pts : \mathcal{V} \rightarrow \mathcal{P}(O)$ is a map from $v \in \mathcal{V}$ to its points-to set, where O is the set of objects created by memory allocation operations in the program, and $\mathcal{P}(O)$ is the power set of O.

Andersen's analysis dynamically inserts new CopyEdges and updates the points-to sets to reach a global fixed point: For each $p \xleftarrow{\text{addr}} o$, we insert o into $pts(p)$; for each $p \xleftarrow{\text{copy}} q$, we merge $pts(q)$ into $pts(p)$; for each $p \xleftarrow{\text{load}} q$ and for each $o \in pts(q)$, we add a new $p \xleftarrow{\text{copy}} o$ to the constraint graph; for each $p \xleftarrow{\text{store}} q$ and for each $o \in pts(p)$, we add a new $o \xleftarrow{\text{copy}} q$ to the constraint graph; and for each $p \xleftarrow{f} q$ and for each $o \in pts(q)$, we insert $o.f$ into $pts(p)$.

In practice, an important optimization for Andersen's PTA is to compute the **strongly connected components (SCCs)** in the constraint graph. Since all nodes in the same cycle consisting of CopyEdges are guaranteed to have identical points-to sets, all these nodes can be collapsed into a single node, known as the representative node [12, 17, 33, 46]. Therefore, the points-to sets of all nodes in the SCC can be updated together.

2.2 MRA and Function-Summary-Based Analysis

Given a function f, the MRA [62, 63] determines the Mod set $\mathcal{D}$ and the Ref set $\mathcal{U}$ of the nonlocal memory locations (Definition 2.3) in f that may be modified and referenced when f is executed.

Definition 2.3 (Local Location and Nonlocal Location). Consider a memory location o that is not represented by a global variable but accessed in a function f. We say o a *local location* under two conditions: (1) o is locally declared within f, and (2) f does not appear in any recursion cycle. Otherwise, o is classified as a *nonlocal location*. We write $Local_f$/$NonLocal_f$ to represent the set of all local/nonlocal locations accessed in f.

Table 2 presents the computation rules for the Mod set and Ref set of different types of statements (Address, Copy, Phi, Offset, Load, Store, and Call) as well as functions (Procedure). The root causes for the inter-procedural side effects (i.e., Mod set $\mathcal{D}$ and Ref set $\mathcal{U}$) in a function f are Loads and Stores [63]. For a Load statement $p = *q$, $pts(q)$ may contain nonlocal locations referenced in f. Similarly, the rule for Store collects nonlocal locations in $pts(p)$ that may be modified at a Store $*p = q$. In contrast, Address, Copy, Phi, and Offset statements do not contribute any side effect. Therefore, the Mod set and Ref set generated by these statements are both empty sets. For a call site cs in a function, the set of nonlocal locations passed into cs that may be referenced or modified inside the callee function, $E(cs)$ (commonly referred to as the escaped object set [61]), is computed and used to intersect with the callees' Mod set and Ref set to filter out spurious Mod/Ref information that propagated from the callees to the caller at this call site. The rule for Procedure indicates that the Mod set and Ref set for the function are the union of the Mod sets and Ref sets of all the statements within f. If a program has recursions, the rules for Call and Procedure are recursively applied until a fixed point is reached.

Table 2. The Rules for Computing the Mod Set and Ref Set of a Function and Various Statements

Type	Form	Mod Set: $\mathcal{D}$	Ref Set: $\mathcal{U}$
Address	$s : p = \textbf{alloca}\ o$	$\mathcal{D}(s) = \emptyset$	$\mathcal{U}(s) = \emptyset$
Copy	$s : p = q$	$\mathcal{D}(s) = \emptyset$	$\mathcal{U}(s) = \emptyset$
Phi	$s : p = \phi(p_1, p_2, \ldots)$	$\mathcal{D}(s) = \emptyset$	$\mathcal{U}(s) = \emptyset$
Offset	$s : p = \&q.f$	$\mathcal{D}(s) = \emptyset$	$\mathcal{U}(s) = \emptyset$
Load	$s : p = *q$	$\mathcal{D}(s) = \emptyset$	$\mathcal{U}(s) = \{o \mid o \in pts(q) \wedge o \in NonLocal_f\}$
Store	$s : *p = q$	$\mathcal{D}(s) = \{o \mid o \in pts(p) \wedge o \in NonLocal_f\}$	$\mathcal{U}(s) = \emptyset$
Procedure	$s_1, ..., s_n$ are the statements in f	$\mathcal{D}(f) = (\bigcup_{i \in [1,n]} \mathcal{D}(s_i))$	$\mathcal{U}(f) = (\bigcup_{i \in [1,n]} \mathcal{U}(s_i))$
Call	$cs : _ = \mathcal{F}_k(_)$	$\mathcal{D}(cs) = (\bigcup_{g_i \in Calee(\mathcal{F}_k)} \mathcal{D}(g_i)) \cap E(cs)$	$\mathcal{U}(cs) = (\bigcup_{g_i \in Calee(\mathcal{F}_k)} \mathcal{U}(g_i)) \cap E(cs)$

MRA is an instance of FSA [24]. The core idea behind FSA is to abstract the behavior of functions in a way that can be reused. This abstraction, or "summary" (denoted as $\mathcal{S}$ in this article), captures the essential effects of a function, such as its side effects on global variables, the heap, or its parameters, as well as its return value. For example, in the case of MRA, the summary computed for function f, denoted as $\mathcal{S}(f)$, consists of the function's Mod set $\mathcal{D}(f)$ and Ref set $\mathcal{U}(f)$. FSA computes and propagates the summary of each function on the call graph, iterating until a fixed point is reached.

3 Motivating Example

This section provides our motivating examples. In particular, we first introduce existing incremental methods for both PTA and MRA and point out their shortcomings. Additionally, we explain how our methods address the weaknesses of the existing incremental PTA and MRA.

3.1 Incremental PTA

Liu et al. [33] propose IPA, an Andersen-style incremental context- and flow-insensitive PTA for Java programs. Compared to previous techniques, IPA fully exploits the transitive nature of Andersen's PTA and reduces a significant amount of redundant computation, substantially enhancing the performance of incremental PTA.

However, we find that in some common cases, IPA is inefficient. Let us take the code deletion as an example. When performing incremental PTA in scenarios of code deletion, two crucial aspects must be addressed. One is the maintenance of SCCs, which involves detecting whether an SCC has been disrupted due to the removal of edges corresponding to the deleted code, and, restoring the nodes and edges within the disrupted SCC to the constraint graph. The second crucial aspect is updating the points-to set of a pointer when changes are propagated to it, and managing the subsequent propagation of the changes. Next, we discuss in detail the shortcomings of IPA in handling these two aspects and present our solutions.

The Maintenance of SCCs. To maintain SCCs, IPA runs Tarjan's algorithm [67] on the impacted SCC containing the deleted edge to detect if the SCC is corrupted each time an edge is removed. This method is inefficient when multiple edge deletions occur within the same SCC. As shown in Figure 3, when the store statement is deleted, the three edges e_1, e_2, and e_3, which are derived from the deleted store statement, need to be removed from the constraint graph. All three edges are part of an SCC in the graph, which means that IPA would need to run Tarjan's algorithm three times on this SCC.

In contrast, our method is more efficient. Specifically, when each edge is removed, we first collect it instead of "eagerly" running SCC detection for every deleted edge. Once all the deleted edges are collected, we then run Tarjan's algorithm on the SCCs containing these edges to determine if the SCC has been disrupted. By delaying the timing of detection, our approach reduces the number of SCC detections needed, which undoubtedly enhances efficiency.

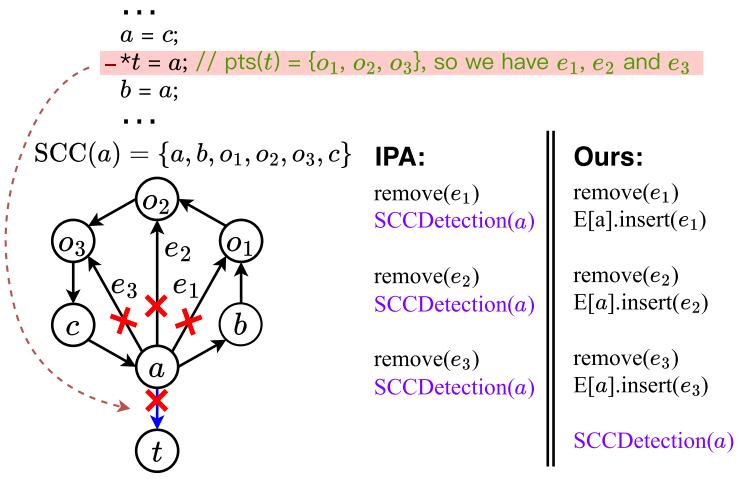


Fig. 3. The difference between IPA and our approach when processing SCC's edge deletion.

The Propagation and Update of Points-To Set Changes. In addition, IPA has drawbacks in the way it propagates and updates the points-to set changes across the constraint graph. When changes propagate to a pointer through a constraint edge, IPA immediately processes this pointer and instantly propagates its changes further. When a pointer is affected by multiple changes along different paths, IPA's "eager" approach can result in numerous updates to the points-to set and frequent propagation of subsequent changes.

Figure 4 illustrates the update and propagation process of the points-to sets by IPA and our method in scenarios that include three changes. Let's first consider Change 1: When the removal of o_1 propagates to p, IPA "eagerly" filters (i.e., determines whether o_1 truly needs to be removed from $pts(p)$), applies (i.e., removes $o1$ from p's points-to set), and propagates (i.e., passes the changes in $pts(p)$ to the subsequent affected pointer q). This is similarly adopted for p's successor q, and q's successor r (as indicated by the blue dashed arrows). Similarly, when handling Change 2, the removal of o_2 also starts from p and follows the same path for propagation (as indicated by the orange dashed arrows). When Change 3 arrives, q and r are processed again in the same manner (as indicated by the green dashed arrows). When the constraint graph is large and complex, the efficiency of IPA processing changes in this manner is compromised—potentially thousands of nodes may need to be repeatedly processed due to multiple arrivals of changes.

In contrast, our method employs a merge-then-propagate approach to avoid redundant processing of the same node. As illustrated, we first merge Change 1 and Change 2 arriving at node p into $\Delta[p]$, allowing p to uniformly filter, apply, and propagate these changes. Similarly, the changes propagated from p to its successor q, along with Change 3, are merged before processing q. It is important to note that the constraint graph is acyclic,[1] allowing us to process all nodes linearly in topological order on each iteration.

3.2 Incremental Function-Summary-Based Analysis

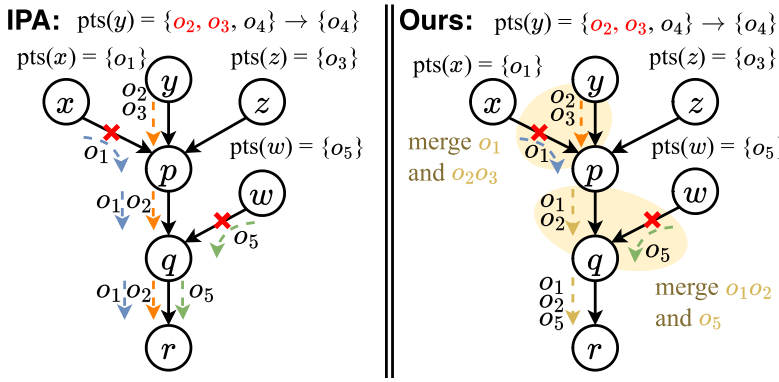
To incrementally update function summaries, existing methods [2, 49, 56, 57, 66, 80] typically rely on the classic reset-recompute approach [16]: Upon detecting changes, they first reset the summaries of all functions "relevant" to the change, and then recompute these summaries according

[1]We always collapse all nodes in SCCs before processing the points-to set changes.

```
f1() {
  // suppose pts(x) = {o1}
  /* Change 1:
   * Delete 'f4(x)' so that
   * edge(x, p) is removed */
- f4(x); // edge(x, p)
}
f2() {
  /* Change 2:
   * Some code changes
   * make pts(y) change from
   * {o2, o3, o4} to {o4} */
  f4(y); // edge(y, p)
}
f3() {
  // suppose pts(z) = {o3}
  f4(z); // edge(z, p)
}
```

```
f4(p) {
  ...
  f5(p); // edge(p, q)
  ...
}
f5(q) {
  ...
  /* Change 3:
   * Delete 'q = w' so that
   * edge(w, q) is removed */
  // Suppose pts(w) = {o5}
- q = w; // edge(w, q)
  r = q; // edge(q, r)
  ...
}
```

(a) A simplified code example with three code changes. Some code content (such as types) that is not relevant to the analysis in this article has been omitted.



(b) The difference between IPA and our approach when updating points-to sets across the constraint graph with three changes in Figure 4a.

Fig. 4. An example of code and constraint graph in incremental PTA.

to the rules used in a full computation. Here, "relevant" functions refer to the functions where the change occurs and all functions that can be reached from the changed functions.

Consider the call graph example shown in Figure 5(b), where rectangular nodes represent functions and directed edges depict the call relationships between functions (for instance, g_1 is a caller of both f_1 and f_2). A change occurs in function f_2, causing its function summary to change from $\{o_1, o_2\}$ to $\{o_1, o_3\}$ (assuming that in this example, the computed function summary represents the function's Ref set), as indicated in Figure 5(a). The reset-recompute algorithm first resets the summaries of f_2 and all functions reachable to f_2 (i.e., g_1 and h_1) to empty. Subsequently, the summary of f_2 is recomputed. Then, in a bottom-up manner, the summary of g_1 is recomputed based on the summaries of f_1 and f_2. Similarly, the summary of h_1 is recomputed based on the summaries of g_1 and g_2. When changes occur in functions at the lower levels of the call graph or when function summaries contain a large amount of data, the reset-recompute algorithm is inefficient. This is because the summaries of a considerable number of functions are reset, and a large volume of summary elements require re-propagation and recomputation.

In contrast, our method propagates only the changed elements within the summary (removing reduced elements and adding new ones). For example, we propagate the changes in f_2's summary to its callers along the call relationship. The caller's summary first clears the reduced element o_2,

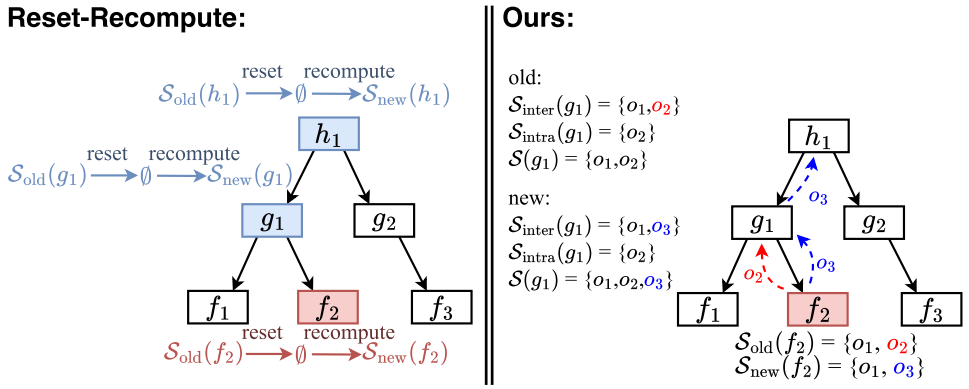
```
h1() {
  g1();
  g2();
}

g1() {
  a = *p; // p points-to o2
  /* The dereference of p accesses
   * the memory object o2,
   * resulting in the intra-summary
   * of g1 being {o2}. */
  f1();
  f2();
}

g2() {
  f3();
}
```

```
f1() {
  ...
}
f2() {
   b = *p; // p points-to o2
-  c = *q; // q points-to o1
+  c = *r; // r points-to o3
   /* Code changes here
   * make f2's summary
   * change from {o1, o2}
   * to {o2, o3}. */
   ...
}
f3() {
  ...
}
```

(a) A simplified code example with some changes in function "f2". Some code content (such as types) that is not relevant to the analysis in this article has been omitted.



(b) The difference between the reset-recompute algorithm and our approach when updating function summary incrementally with the changes in Figure 5a. The red node represents the function where change occurs. $\mathcal{S}$ represents the function summary, while $\mathcal{S}_{intra}$ and $\mathcal{S}_{inter}$ represent the intra-procedural and inter-procedural summaries of the function, respectively. The subscripts "old" and "new" refer to the old and new versions of the function summary. Red elements represent deleted summary elements, while blue elements represent newly added summary elements.

Fig. 5. An example of code and call graph in incremental function-summary-based analysis.

then adds the new element o_3. In addition, we use intra-procedural summary (denoted as $\mathcal{S}_{intra}$) to filter the propagated changed element in inter-procedural summary (denoted as $\mathcal{S}_{inter}$), allowing the summary update process to stop at lower levels of the call graph. For instance, since g_1's intra-procedural summary includes o_2, o_2 will not be removed from g_1's summary, resulting in stopping o_2's propagation at g_1.

4 Methodology

In this section, we detail our incremental SVFA. We begin with an overview of our method, followed by a detailed description of the two incremental modules—incremental PTA and incremental MRA.

4.1 Overview of SILVA

Building upon the three-staged full SVFA introduced earlier in Figure 1, we present the workflow of our incremental SVFA.

First, SILVA receives the program changes as input. The first incremental module, IncPTA, solves the points-to constraint by updating the constraint graph incrementally to obtain the new points-to information and the new call graph. Compared to the full method, which rebuilds the entire constraint graph and computes the points-to information from scratch, our method only incrementally updates the constraint graph and the points-to set affected by program changes, making it more efficient. Subsequently, from the IncPTA module, we obtain the changes of the program, points-to information, and call graph, all of which serve as input for the next module, namely IncMRA. IncMRA identifies the functions where changes have occurred. It then begins from the nodes of these functions in a bottom-up fashion, utilizing our incremental algorithm to update the Mod-Ref information for each function. Subsequent modules re-perform memory region partitioning, memory SSA construction, and value-flow construction based on the new Mod-Ref information. Note that for large-scale programs, the time spent on the three modules typically accounts for less than 10% of the overall time. So our incremental approach targeting bottleneck modules can effectively improve the overall efficiency of SVFA.

4.2 Incremental PTA

We now introduce our incremental PTA. Our incremental PTA is designed for C/C++ pointer constraints, and is flow- and context-insensitive, matching the precision of the most commonly used full PTA in SVFA. For brevity of description, we cover the details of the algorithm in the context of field-insensitive. However, our incremental algorithm can be extended to field-sensitive PTA (as we implemented for experiments) by adding another type of constraint (i.e., offset constraint in Table 1) similar to the previous work [45].

The difficulty of incremental PTA lies in the processing of incremental statement deletion [31–33, 80]. Therefore, we focus on how to handle statement deletions, while the procedure of resolving incremental statement insertion is similar but simpler.

4.2.1 Incremental Edge Deletion: Brief Outline. At a high level, our incremental PTA and the state-of-the-art incremental PTA algorithm, IPA [31, 33], follow similar steps, namely: (1) extracting and removing constraint edges from the constraint graph based on the deleted statements, (2) detecting whether any SCCs are disrupted due to the removal of constraint edges, and (3) propagating and updating changes in the points-to set caused by the deletions.

Due to changes in the points-to set and the complex constraints (i.e., store and load), further constraint edge deletions may occur. IPA utilizes a unified worklist to process each constraint edge that needs to be removed in each iteration according to the three steps mentioned above. However, our approach distinguishes itself by utilizing two maps to separate these three steps, creating three distinct but interrelated parts. Our key insight is that prioritizing the collection and merging of changes to the same object and processing them collectively in subsequent steps can reduce redundant operations, as discussed in Section 3.1. We use the map $\mathcal{R}$ to collect deleted edges within each SCC, and the map Δ to merge changes in the points-to sets that propagate to the same pointer. The specific definitions of these two mappings are as follows:

- $\mathcal{R} : \mathcal{V} \mapsto \mathcal{P}(\mathcal{E})$. A map from an SCC representative node to the set of removed edges in that SCC. Our approach associates multiple edges inside the same SCC with the representative node of the SCC, and performs SCC broken detection on each collected representative node when appropriate, avoiding frequent SCC detection computation.
- $\Delta : \mathcal{V} \mapsto \mathcal{P}(\mathcal{O})$. A map from a node to the set of objects that are potentially removed from the node's points-to set. Our approach associates a node with all the objects potentially deleted from its points-to set, so that the points-to set can be uniformly filtered, updated, and propagated when working with the node, avoiding repeated processing of the same node.

$$\textbf{DelAddr}\ \frac{\begin{array}{c} e_A = p \xleftarrow{\text{addr}} o \in \mathcal{E} \\ r = rep(p) \end{array}}{\begin{array}{l} \mathcal{E} \leftarrow \mathcal{E} \backslash \{e_A\} \\ \Delta(r) \leftarrow \Delta(r) \cup \{o\} \end{array}}$$

$$\textbf{DelCopy}\ \frac{\begin{array}{c} e_C = p \xleftarrow{\text{copy}} q \in \mathcal{E} \\ r_1 = rep(p) \quad r_2 = rep(q) \end{array}}{\begin{array}{l} \mathcal{E} \leftarrow \mathcal{E} \backslash \{e_C\} \\ \begin{cases} r_1 \neq r_2 : \Delta(r_1) \leftarrow \Delta(r_1) \cup pts(r_2) \\ r_1 = r_2 : \mathcal{R}(r_1) \leftarrow \mathcal{R}(r_1) \cup \{e_C\} \end{cases} \end{array}}$$

$$\textbf{DelLoad}\ \frac{\begin{array}{c} e_L = p \xleftarrow{\text{load}} q \in \mathcal{E} \\ O = pts(q) \end{array}}{\begin{array}{l} \mathcal{E} \leftarrow \mathcal{E} \backslash \{e_L\} \\ \textbf{DelCopyByLoad}(e_L, O) \end{array}}$$

$$\textbf{DelStore}\ \frac{\begin{array}{c} e_S = p \xleftarrow{\text{store}} q \in \mathcal{E} \\ O = pts(p) \end{array}}{\begin{array}{l} \mathcal{E} \leftarrow \mathcal{E} \backslash \{e_S\} \\ \textbf{DelCopyByStore}(e_S, O) \end{array}}$$

$$\textbf{DelCopyByLoad}\ \frac{\begin{array}{c} e_L = p \xleftarrow{\text{load}} q \quad o \in O \\ e_C = p \xleftarrow{\text{copy}} o \in \mathcal{E} \\ r_1 = rep(p) \quad r_2 = rep(o) \end{array}}{\begin{array}{l} \mathcal{E} \leftarrow \mathcal{E} \backslash \{e_C\} \\ \begin{cases} r_1 \neq r_2 : \Delta(r_1) \leftarrow \Delta(r_1) \cup pts(r_2) \\ r_1 = r_2 : \mathcal{R}(r_1) \leftarrow \mathcal{R}(r_1) \cup \{e_C\} \end{cases} \end{array}}$$

$$\textbf{DelCopyByStore}\ \frac{\begin{array}{c} e_S = p \xleftarrow{\text{store}} q \quad o \in O \\ e_C = o \xleftarrow{\text{copy}} q \in \mathcal{E} \\ r_1 = rep(o) \quad r_2 = rep(q) \end{array}}{\begin{array}{l} \mathcal{E} \leftarrow \mathcal{E} \backslash \{e_C\} \\ \begin{cases} r_1 \neq r_2 : \Delta(r_1) \leftarrow \Delta(r_1) \cup pts(r_2) \\ r_1 = r_2 : \mathcal{R}(r_1) \leftarrow \mathcal{R}(r_1) \cup \{e_C\} \end{cases} \end{array}}$$

Fig. 6. The inference rules of handling a deleted constraint edge.

4.2.2 Incremental Edge Deletion: Algorithm and Basic Rules. We now present our algorithm and go through the parts in detail. We first present the basic inference rules for handling edge deletions that are easily explainable. Subsequently, in the next section, we expand these rules to support complex edge deletion scenarios that are not supported by IPA.

Part 1: Constraint Edge Deletion. This part traverses each deleted edge (i.e., *WL*) and processes the edge by its edge type (lines 3 to 5). Figure 6 depicts basic rules for edge deletion.

We call rule **DelAddr** to remove an AddrEdge $e_A = p \xleftarrow{\text{addr}} o$ generated by a deleted address statement. The $rep(p)$ operation returns the representative node r of the SCC where p is located. Subsequently, e_A will be removed directly from the edge set $\mathcal{E}$ of the constraint graph, and o, the memory object allocated by the address statement s, will be stored in $\Delta(r)$ as a potential changed object of $pts(r)$.

Rule **DelCopy** is applied to remove a CopyEdge $e_C = p \xleftarrow{\text{copy}} q$ generated by a deleted copy statement, phi statement, or call statement. First, we remove e_C from $\mathcal{E}$ and handle it differently depending on whether e_C is inside an SCC or not:

- If e_C is inside the SCC (i.e., $rep(p) = r_1 = r_2 = rep(q)$), we detect whether the SCC represented by r_1 is broken into multiple smaller SCCs or individual nodes due to the removal of e_C. Note, however, that instead of immediately executing the SCC detection algorithm, we temporarily store e_C in $\mathcal{R}(r_1)$, and only in the second part of the algorithm do we detect the SCCs stored in $\mathcal{R}$.
- If e_C is not inside any SCC (i.e., $rep(p) = r_1 \neq r_2 = rep(q)$), then when e_C is removed, we need to store $pts(r_2)$ as potential points-to set changes of r_1. Because after deleting the edge, all objects in $pts(r_2)$ may be removed from $pts(r_1)$.

We call rule **DelLoad** to remove a LoadEdge $e_L = p \xleftarrow{\text{load}} q$ generated by a deleted load statement. Note that the load constraint is a complex constraint that introduces some derived CopyEdges to the constraint graph. For a LoadEdge $e_L = p \xleftarrow{\text{load}} q$, the derived CopyEdge takes the object o in the $pts(q)$ as the source node and p as the destination node. Therefore, we then call rule **DelCopyByLoad** on e_L and $pts(q)$ to further handle the deletion of such derived CopyEdges. Rule **DelCopyByLoad** collects all CopyEdges that start with an object in O and end with the

$$\textbf{SCCBroken}\quad \frac{\begin{array}{c} r \in \mathcal{R}.KeySet \quad SCC(r) \text{ is broken} \\ e_C = p \xleftarrow{\text{copy}} q \in \mathcal{R}(r) \\ r_1 = rep(p) \quad r_2 = rep(q) \quad r_1 \neq r_2 \end{array}}{\Delta(r_1) \leftarrow \Delta(r_1) \cup pts(r_2)}$$

$$\textbf{FilterDeletion}\quad \frac{\begin{array}{c} \Delta(p) \neq \emptyset \quad r = rep(p) \\ \mathcal{W} = \{w \mid p' \xleftarrow{\text{copy}} w \in \mathcal{E} \wedge r = rep(p') \wedge r \neq rep(w)\} \\ \mathcal{V} = \{v \mid p' \xleftarrow{\text{addr}} v \in \mathcal{E} \wedge r = rep(p')\} \\ \Delta_{IN} = (\bigcup_{w_i \in \mathcal{W}} pts(w_i)) \cup \mathcal{V} \\ \Delta_{diff} = \Delta(r) \backslash \Delta_{IN} \quad \Delta_{diff} \neq \emptyset \end{array}}{\begin{array}{c} pts(r) \leftarrow pts(r) \backslash \Delta_{diff} \\ \textbf{PropagateDeletion}(r, \Delta_{diff}) \end{array}}$$

$$\textbf{PropagateDeletion}\quad \frac{\begin{array}{c} r \text{ is an SCC representative node} \quad \Delta_{diff} \\ \mathcal{W} = \{w \mid w \xleftarrow{\text{copy}} p \in \mathcal{E} \wedge r \neq rep(w) \wedge r = rep(p)\} \\ \mathcal{X} = \{x \mid x \xleftarrow{\text{load}} p \in \mathcal{E} \wedge r = rep(p)\} \\ \mathcal{Y} = \{y \mid p \xleftarrow{\text{store}} y \in \mathcal{E} \wedge r = rep(p)\} \end{array}}{\begin{array}{c} \forall w \in \mathcal{W} : \Delta(w) \leftarrow \Delta(w) \cup \Delta_{diff} \\ \forall x \in \mathcal{X} : \textbf{DelCopyByLoad}(x \xleftarrow{\text{load}} p, \Delta_{diff}) \\ \forall y \in \mathcal{Y} : \textbf{DelCopyByStore}(p \xleftarrow{\text{store}} y, \Delta_{diff}) \end{array}}$$

Fig. 7. The inference rules of SCC broken detection and points-to set changes propagation.

destination node p of e_L, and applies the same way as the **DelCopy** depending on whether e_C is in SCC. Similarly, rules **DelStore** and **DelCopyByStore** handle the deletion of StoreEdge and its derived CopyEdges.

Part 2: SCC Broken Detection. In Part 2, a key-value pair $<r, E_r>$ in $\mathcal{R}$ will be fetched in each iteration. The key r is the representative node of an SCC, and the value E_r is the collected CopyEdges that have been deleted from that SCC. We then call `SCCBrokenDetection` (line 11) to detect whether the SCC represented by r is broken into multiple smaller SCCs or individual nodes due to the removal of edges in E_r. Specifically, the SCC detection algorithm used here is an improved version of Tarjan's algorithm [67], as proposed in [43]. This method runs in linear time on the number of edges and, as pointed by Pearce et al. [45], Pereira and Berlin [46], it has the beneficial side effect of producing a topological ordering of the target graph for free. Leveraging this property, we can update the topological order of nodes in the constraint graph with almost no overhead, facilitating efficient points-to set updates and propagation in Part 3.

If an SCC is not broken, we keep the representative node and maintain the rest of the SCC structure. Otherwise, we follow rule **SCCBroken** listed in Figure 7 to update changes to the points-to set.

Rule **SCCBroken** processes the deleted CopyEdges that cause an SCC to be broken. Recall that when we handle the deletion of a CopyEdge (refer to rule **DelCopy** and **DelCopyByLoad/Store** in Figure 6), if two nodes of the CopyEdge are in the same SCC, we just simply record it in $\mathcal{R}$, and the points-to set changes impacted by the deleted CopyEdge are not handled. Rule **SCCBroken** first obtains r_1 and r_2, which are the representative nodes of the new SCCs where the source and destination of CopyEdge are located, respectively. Then, $pts(r_2)$ is stored in $\Delta(r_1)$ as a potential points-to set changes of r_1.

Part 3: Points-To Set Updating and Propagation. This part updates and propagates the points-to set changes through the acyclic constraint graph by Rule **FilterDeletion** and **PropagateDeletion**. First, we place all the nodes of the constraint graph in $\mathcal{N}_T$ according to their topological order. Since we performed SCC detection and updated the topological order of the nodes in Part 2, there is no need for additional overhead to re-sort the nodes topologically. Subsequently, we will update the points-to sets of each node in topological order, if the pointer changes propagated to that node are non-empty. When a node p's potential points-to set changes $\Delta(p)$ is taken out of Δ, rule **FilterDeletion** gets the real change Δ_{diff} by filtering out the objects which are the incoming AddrEdge neighbors of SCC(p),[2] and the objects in the points-to-set of neighbors of all incoming CopyEdges of SCC(p). The intuition of rule **FilterDeletion** is based on the Incoming and Outgoing Neighbors Property proposed by IPA [33], that is, when a potentially deleted element propagates along an edge to a node, if the node still has other incoming edges that ensure the existence of that element in the node's points-to set, then the element should be retained in the points-to set of that node. Interested readers may refer to Theorems 1 and 2 in Section 4 of the IPA paper [33].

After updating the points-to set of the representative node, we then call rule **PropagateDeletion** to propagate the change, Δ_{diff}, to all outgoing CopyEdge neighbors u. In addition, the points-to set of SCC(r) is changed, which affects the derived CopyEdges of SCC(r)'s outgoing LoadEdges and incoming StoreEdges. So we need to delete them by applying rule **DelCopyByLoad** and rule **DelCopyByStore**, respectively. Note that the new deletion of the CopyEdge causes a change in $\mathcal{R}$, which means that we end up looping back to Part 2.

We then update the call graph based on changes in the points-to sets of function pointers (line 18). $\mathcal{P}_{\text{func}}$ denotes the set of pts-changed-function-pointers. We first locate the call sites associated with the pts-changed-function-pointers. Note that the element to which a function pointer points is a function object, that is, the callee of the indirect call site. For each callee to be deleted at the indirect call site,[3] we remove the CopyEdges corresponding to the parameter and return value passing by applying rule **DelCopy**. Then, the indirect call edges from the call sites to those deleted callees are removed from the call graph.

Finally, before the loop ends, the algorithm checks if $\mathcal{R}$ and Δ have changed to decide whether to reanalyze. When $\mathcal{R}$ and Δ are both empty after a certain iteration, the analysis reaches the fixed point and the algorithm ends.

4.2.3 Incremental Edge Deletion: Extended Rules. The above are our basic rules for dealing with deletion, now we move to a more complicated case.

Figure 8 shows an example where the basic rules do not apply. The CopyEdge from c_1 is derived by both s_1 and s_2, as a copy path from x to y makes the two destination nodes have the same points-to set $\{o\}$. If the StoreEdge s_1 is removed from the constraint graph due to the deletion of the program statement $*x = a$ (as shown in Figure 2), we will call rule **DelCopyByStore**(s_1, $\{o\}$) to remove the CopyEdge c_1, which may cause a difference between $pts(a)$ and $pts(o)$. However, the other StoreEdge s_2 is *not* deleted, which means the derived CopyEdge c_1 should remain in the constraint graph.

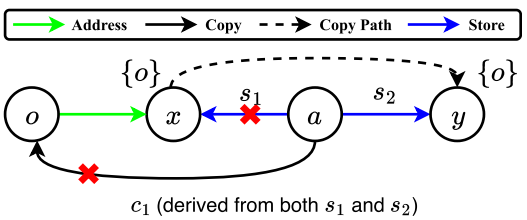


Fig. 8. An example of a CopyEdge removal error.

[2]We use SCC(p) to represent all nodes in the SCC where p resides.
[3]It only means the removal of the call relationship from the call site to the callee, instead of the deletion of the call site.

Therefore, hastily removing CopyEdges derived by LoadEdges or StoreEdges can lead to wrong results. Such issues only arise with CopyEdges, as they are the only ones derived from other edges during analysis. Other types of edges (i.e., AddrEdge, LoadEdge, and StoreEdge) correspond directly to program statements, so when these edges need to be removed due to the deletion of program statements, we can directly remove them from the constraint graph.

Some incremental PTA methods [48, 80] based on the reset-recompute approach can address the aforementioned CopyEdge deletion issue. Taking Figure 8 as an example, during the reset phase, these methods remove the derived c_1 due to the removal of s_1. Then, in the recomputation, c_1 is re-added because of the presence of s_2. However, as pointed out by recent research [31–33] on Andersen-style incremental PTA, which exploits an insight on the fundamental transitivity property of fixed-point-based PTA, the reset-recompute methods lead to a significant amount of redundant computation (e.g., c_1 being redundantly deleted and re-added). This inefficiency makes the reset-recompute methods unscalable when dealing with code deletions. Unfortunately, existing state-of-the-art methods [31–33] based on transitivity property only address the redundant computation of the points-to set elements in reset-recompute approaches, while overlooking the issues caused by the multi-source derivations of CopyEdge as described above.

To solve this problem, we maintain a map $\mathcal{B}$ to record the binding relationship between CopyEdge and other types of constraint edges:

— $\mathcal{B} : \mathcal{E}_{\text{copy}} \mapsto \mathcal{P}(\mathcal{E})$—A map from a CopyEdge to the set of edges that derive the CopyEdge. It is maintained during the processing of CopyEdge insertions or deletions in PTA. Specifically, if a CopyEdge c corresponds to a statement in the program (i.e., Copy, Phi, or Call as defined in Table 1), or in other words, is generated by a program statement, we place c into its own binding set (i.e., $c \in \mathcal{B}(c)$). This definition of the binding set fully accounts for all possible sources of a CopyEdge. For example, before any edge is removed, the binding set of the CopyEdge c_1 in Figure 8 is

$$\mathcal{B}(c_1) = \begin{cases} \{s_1, s_2, c_1\}, & \text{if } c_1 \text{ corresponds to a program statement;} \\ \{s_1, s_2\}, & \text{otherwise.} \end{cases}$$

We expanded several of the basic rules for dealing with CopyEdge, as shown in Figure 9. In contrast to the basic ones, the first three expanded rules that handle CopyEdge removal call rule **Unbind** to maintain the binding set instead of removing the CopyEdge directly. Then rule **Unbind** unbinds the derived CopyEdge e_C to the edge from which e_C is derived. If the binding set $\mathcal{B}(e_C)$ of a CopyEdge e_C is empty, meaning that e_C is not derived from any other constrained edge, then rule **EmptyBind** will remove e_C in the same way that CopyEdge deletions are handled in the basic rules, i.e., update $\mathcal{R}$ or Δ according to whether e_C is inside an SCC.

Let's consider the example in Figure 8 again. Suppose c_1 does not correspond to any program statement, then $\mathcal{B}(c_1) = \{s_1, s_2\}$. When a store statement $*x = a$ is deleted in the updated program, we extract the corresponding StoreEdge s_1 and remove it from the constraint graph, and then we call the extended rule **DelCopyByStore** to unbind the CopyEdge c_1 and the StoreEdge s_1 which c_1 derived by, resulting in $\mathcal{B}(c_1) = \{\cancel{s_1}, s_2\}$. The final $\mathcal{B}(c_1)$ is not empty, which means that c_1 is still bound to some edge, so c_1 will remain in the constraint graph, which is the correct analysis result.

4.2.4 Incremental Edge Deletion: Correctness. Now we prove the correctness of our incremental edge deletion algorithm.

Theorem 4.1 (Correctness of Incremental Edge Deletion Algorithm). *Our incremental edge deletion algorithm is correct because, for an original program P and another program P^- resulted from the deletion of the edges C_D in P, applying our incremental deletion algorithm to C_D on the full*

$$\textbf{DelCopy}\quad \frac{e_C = p \xleftarrow{\text{copy}} q \in \mathcal{E}}{\textbf{Unbind}(e_C, e_C)}$$

$$\textbf{DelCopyByLoad}\quad \frac{e_L = p \xleftarrow{\text{load}} q \quad o \in O \qquad e_C = p \xleftarrow{\text{copy}} o \in \mathcal{E}}{\textbf{Unbind}(e_C, e_L)}$$

$$\textbf{DelCopyByStore}\quad \frac{e_S = p \xleftarrow{\text{store}} q \quad o \in O \qquad e_C = o \xleftarrow{\text{copy}} q \in \mathcal{E}}{\textbf{Unbind}(e_C, e_S)}$$

$$\textbf{Unbind}\quad \frac{e_C \in \mathcal{E} \quad e}{\mathcal{B}(e_C) \leftarrow \mathcal{B}(e_C) \backslash \{e\}}$$

$$\textbf{EmptyBind}\quad \frac{e_C = p \xleftarrow{\text{copy}} q \in \mathcal{E} \quad \mathcal{B}(e_C) = \emptyset \qquad r_1 = rep(p) \quad r_2 = rep(q)}{\mathcal{E} \leftarrow \mathcal{E} \backslash \{e_C\} \qquad \begin{cases} r_1 \neq r_2 : \Delta(r_1) \leftarrow \Delta(r_1) \cup pts(r_2) \\ r_1 = r_2 : \mathcal{R}(r_1) \leftarrow \mathcal{R}(r_1) \cup \{e_C\} \end{cases}}$$

Fig. 9. The expanded rules of handling a deleted constraint edge. **DelCopy**, **DelCopyByLoad**, and **DelCopyByStore** expand upon the basic rules presented in Figure 6. The expansions are all based on **Unbind**, which is a new rule we introduce (at the bottom left of the figure). **EmptyBind** extends from the basic rule **DelCopy**, adding an inference condition where the binding set $\mathcal{B}(e_C)$ is empty. This ensures that the CopyEdge is deleted only when it no longer binds with other edges.

analysis results $Res(P)$ yields $IncRes_D(Res(P), C_D)$, which are consistent with the full analysis results $Res(P^-)$ of the program after the deletion.

Proof. The analysis results of PTA include the points-to sets of all pointers, a constraint graph, and a call graph of the program. We first prove the consistency of the points-to sets and the constraint graph produced by the incremental algorithm with those produced by the full analysis by demonstrating the satisfaction of Andersen's points-to constraints after the incremental analysis (as guaranteed by Andersen's analysis [1]). Subsequently, since our incremental algorithm modifies the indirect call graph solely based on the points-to sets of function pointers, the correctness of the indirect call graph naturally follows from the correctness of the function pointers' points-to sets.

We will categorize and discuss the satisfaction for different types of constraints after applying the incremental deletion algorithm. The scenarios discussed here involve only non-SCC internal edges, that is, edges that do not belong to an SCC. The correctness of handling deleted edges within SCCs is ensured by the correctness of the SCC detection and maintenance algorithms [67], and thus will not be elaborated further.

— *Address Constraint*: $\forall p \xleftarrow{\text{addr}} o \in \mathcal{E}$, o must be in $pts(p)$. Suppose there is an AddrEdge $e_A = p \xleftarrow{\text{addr}} o$ and $o \notin pts(p)$, then o must have been propagated to p as a potential change points-to set element from p's incoming CopyEdge/AddrEdge neighbors and applied by p, meaning o was removed from $pts(p)$. However, according to the rule **FilterDeletion**, since there exists an AddrEdge e_A, then $o \in \mathcal{V}$, which means o is removed from p's potential change $\Delta(p)$. Therefore, o is not in Δ_{diff} and must be retained in $pts(p)$.

— *Load Constraint*: $\forall p \xleftarrow{\text{load}} q \in \mathcal{E}$, if $o \in pts(q)$, then there must be a CopyEdge $p \xleftarrow{\text{copy}} o \in \mathcal{E}$. Suppose there is a LoadEdge $e_L = p \xleftarrow{\text{load}} q$ and an object $o \in pts(q)$, such that $e_C = p \xleftarrow{\text{copy}} o \notin \mathcal{E}$, then e_C must have been removed from the constraint graph, meaning $\mathcal{B}(e_C)$ had become empty according to the rule **EmptyBind**. However, based on the correctness of the full PTA, since e_L has always been in the constraint graph and was never removed, e_L would always exist in the binding set of e_C, $\mathcal{B}(e_C)$ according to the rules **DelCopyByLoad** and **Unbind**. This indicates that e_C could not have been removed from the constraint graph.

— *Store Constraint*: $\forall p \xleftarrow{\text{store}} q \in \mathcal{E}$, if $o \in pts(p)$, then there must be a CopyEdge $o \xleftarrow{\text{copy}} q \in \mathcal{E}$. Suppose there is a StoreEdge $e_S = p \xleftarrow{\text{store}} q$ and an object $o \in pts(p)$, such that $e_C = o \xleftarrow{\text{copy}} q \notin \mathcal{E}$, then e_C must have been removed from the constraint graph, meaning $\mathcal{B}(e_C)$ had become empty according to the rule **EmptyBind**. However, based on the correctness of the full PTA, since e_S has always been in the constraint graph and was never removed, e_S would always exist in the binding set of e_C, $\mathcal{B}(e_C)$ according to the rules **DelCopyByStore** and **Unbind**. This indicates that e_C could not have been removed from the constraint graph.

— *Copy Constraint*: $\forall p \xleftarrow{\text{copy}} q \in \mathcal{E}, pts(p) \subseteq pts(q)$, that is, $\forall o \in pts(q)$, o must be in $pts(p)$. Suppose there is a CopyEdge $e_C = p \xleftarrow{\text{copy}} q$, and an object $o \in pts(q)$, such that $o \notin pts(p)$, then o must have been propagated to p as a potential change points-to set change element from p's incoming CopyEdge/AddrEdge neighbors and applied by p, meaning o was removed from $pts(p)$. However, according to the rule **FilterDeletion**, since there exists a CopyEdge e_C and $o \in pts(q)$, then $o \in \mathcal{U}$, which means o is removed from p's potential change $\Delta(p)$. Therefore, o is not in Δ_{diff} and must be retained in $pts(p)$. □

4.2.5 Incremental Edge Insertion. Algorithm 2 shows our incremental approach for handling constraint edge insertion. It has a similar structure to our incremental deletion algorithm.

The first part of Algorithm 2 is to iterate over *WL* to insert the newly created constraint edges corresponding to the inserted statements into the constraint graph. Figure 10 shows the insertion rules for the various types of constraint edges. To solve the problem of the multi-source derivation of Figure 8 mentioned earlier, we also maintain the binding set of CopyEdges through the rule **Bind**. When a CopyEdge e_c derived from another edge e is inserted into the constraint graph, the rule **Bind** adds e to e_C's binding set $\mathcal{B}(e_C)$. We propagate the points-to set changes only when a CopyEdge that is new (i.e., does not already exist in the constraint graph) and connects two SCCs (i.e., the two nodes of the CopyEdge have different representative nodes) is inserted into the graph, as shown in rule **NewCopyEdge**. When the CopyEdge already exists in the graph, or when the CopyEdge is inside an SCC, we do not redundantly propagate the points-to set changes.

Since new CopyEdges may be inserted into the graph, the second part of the algorithm then performs SCC cycle detection and collapses the new SCC to keep the constraint graph acyclic. After that, rules **FilterInsertion** and **PropagateInsertion** (Figure 11) are called in Part 3 to update the points-to set changes in the topological order of the nodes through the acyclic graph.

We also update the call graph in Algorithm 2. `UpdateIndirectCallEdge` is applied at line 18. It checks the points-to set changes of the function pointers to compute the new indirect call relationship and insert the new call edge into the call graph. This is similar to the approach in the algorithm for handling deletion.

Finally, the algorithm terminates when there are no edges to be inserted or no new points-to set changes to be propagated.

Theorem 4.2 (Correctness of Incremental Edge Insertion Algorithm). *Our incremental edge insertion algorithm is correct because, for two programs P and P^+ before and after an insertion change, and for all inserted edges C_I, applying our incremental insertion algorithm to C_I on the full analysis results $Res(P)$ yields analysis results $IncRes(Res(P), C_I)$ that are consistent with the full analysis results $Res(P^+)$ of the program after the insertion.*

Proof. We follow the proof method outlined in Theorem 4.1, where we classify and discuss the satisfaction of different types of constraints after applying the incremental algorithm. This is done to prove that the points-to sets and the constraint graph produced by the incremental algorithm are

$$\textbf{InsAddr}\ \frac{e_A = p \xleftarrow{\text{addr}} o \notin \mathcal{E} \quad rep(p) = r}{\mathcal{E} \leftarrow \mathcal{E} \cup \{e_A\} \quad \Delta(r) \leftarrow \Delta(r) \cup \{o\}} \qquad \textbf{InsCopy}\ \frac{e_C = p \xleftarrow{\text{copy}} q}{\textbf{Bind}(e_C, e_C)}$$

$$\textbf{InsLoad}\ \frac{e_L = p \xleftarrow{\text{load}} q \notin \mathcal{E} \quad O = pts(q)}{\mathcal{E} \leftarrow \mathcal{E} \cup \{e_L\} \quad \textbf{InsCopyByLoad}(e_L, O)} \qquad \textbf{InsStore}\ \frac{e_S = p \xleftarrow{\text{store}} q \notin \mathcal{E} \quad O = pts(p)}{\mathcal{E} \leftarrow \mathcal{E} \cup \{e_S\} \quad \textbf{InsCopyByStore}(e_S, O)}$$

$$\textbf{InsCopyByLoad}\ \frac{e_L = p \xleftarrow{\text{load}} q \quad o \in O \quad e_C = p \xleftarrow{\text{copy}} o}{\textbf{Bind}(e_C, e_L)} \qquad \textbf{InsCopyByStore}\ \frac{e_S = p \xleftarrow{\text{store}} q \quad o \in O \quad e_C = o \xleftarrow{\text{copy}} q}{\textbf{Bind}(e_C, e_S)}$$

$$\textbf{Bind}\ \frac{e_C \text{ is derived from } e}{\mathcal{B}(e_C) \leftarrow \mathcal{B}(e_C) \cup \{e\} \quad \textbf{if } e_C \notin \mathcal{E} : \mathcal{E}_{\text{new}} \leftarrow \mathcal{E}_{\text{new}} \cup \{e_C\} \quad \textbf{NewCopyEdge}(e_C)}$$

$$\textbf{NewCopyEdge}\ \frac{e_C = p \xleftarrow{\text{copy}} q \notin \mathcal{E} \quad r_1 = rep(p) \quad r_2 = rep(q)}{\mathcal{E} \leftarrow \mathcal{E} \cup \{e_C\} \quad \textbf{if } r_1 \neq r_2 : \Delta(r_1) \leftarrow \Delta(r_1) \cup pts(r_2)}$$

Fig. 10. The inference rules of handling an inserted constraint edge.

$$\textbf{FilterInsertion}\ \frac{\Delta(p) \neq \emptyset \quad r = rep(p) \quad \Delta_{diff} = \Delta(r) \backslash pts(r) \quad \Delta_{diff} \neq \emptyset}{pts(r) \leftarrow pts(r) \cup \Delta_{diff} \quad \textbf{PrpopagateInsetion}(r, \Delta_{diff})}$$

$$\textbf{PropagateInsertion}\ \frac{\begin{array}{c} r \text{ is an SCC representative node} \quad \Delta_{diff} \\ \mathcal{W} = \{w \mid w \xleftarrow{\text{copy}} p \in \mathcal{E} \wedge rep(w) \neq r \wedge r = rep(p)\} \\ \mathcal{X} = \{x \mid x \xleftarrow{\text{load}} p \in \mathcal{E} \wedge r = rep(p)\} \\ \mathcal{Y} = \{y \mid p \xleftarrow{\text{store}} y \in \mathcal{E} \wedge r = rep(p)\} \end{array}}{\begin{array}{l} \forall w \in \mathcal{W} : \Delta(w) \leftarrow \Delta(w) \cup \Delta_{diff} \\ \forall x \in \mathcal{X} : \textbf{InsCopyByLoad}(x \xleftarrow{\text{load}} p, \Delta_{diff}) \\ \forall y \in \mathcal{Y} : \textbf{InsCopyByStore}(p \xleftarrow{\text{store}} y, \Delta_{diff}) \end{array}}$$

Fig. 11. The inference rules of handling points-to set changed when processing insertion.

consistent with those generated by the full Andersen's algorithm. The correctness of the indirect call graph is naturally ensured by the correctness of the function pointers' points-to sets. Similarly, the scenarios discussed here involve only edges that are not within SCCs.

— *Address Constraint*: $\forall p \xleftarrow{\text{addr}} o \in \mathcal{E}$, o must be in $pts(p)$. Suppose there is an AddrEdge $e_A = p \xleftarrow{\text{addr}} o$ and $o \notin pts(p)$. Then e_A must have been newly added to the constraint graph, and Algorithm 2 fails to insert o into $pts(p)$. This is impossible because, according to the rule of **InsAddr**, it must hold that $o \in \Delta(p)$. Then, in Part 3, since it is assumed that $o \notin pts(p)$, **FilterInsertion** will not filter out o, meaning $o \in \Delta_{diff}$. Then, **FilterInsertion** will merge

Δ_{diff} into $pts(p)$, implying that o will be inserted into $pts(p)$, and thus $o \in pts(p)$. Therefore, the assumption is invalid.

— *Load Constraint*: $\forall p \xleftarrow{\text{load}} q \in \mathcal{E}$, if $o \in pts(q)$, then there must be a CopyEdge $p \xleftarrow{\text{copy}} o \in \mathcal{E}$. Suppose there is a LoadEdge $e_L = p \xleftarrow{\text{load}} q$ and an object $o \in pts(q)$, such that $e_C = p \xleftarrow{\text{copy}} o \notin \mathcal{E}$. Then there are two possible cases: (a) o was inserted into $pts(q)$ due to a certain change, and Algorithm 2 fails to solve the Load constraint related to q based on the change in $pts(q)$; (b) e_L is newly added to the constraint graph, and Algorithm 2 fails to solve this new Load constraint. For cases (a), when Algorithm 2 updates $pts(q)$ in Part 3, according to the rule **FilterInsertion**, o must be in Δ_{diff} at this point, otherwise o would not have been newly inserted into $pts(q)$. Since e_L is a LoadEdge from q to p, according to **PropagateInsertion**(q, Δ_{diff}), we have $p \in \mathcal{X}$, and we also have **InsCopyByLoad**(e_L, Δ_{diff}). Then, based on the rules **InsCopyByLoad**, **Bind**, and **NewCopyEdge**, since $o \in \Delta_{diff}$ (i.e., $o \in \mathcal{O}$), e_C and e_L will be bound, and if e_C does not already exist, Algorithm 2 will add e_C to the graph, thus the assumption is invalid. For case (b), when Algorithm 2 inserts edges in Part 1, according to the rule **InsLoad**, we have **InsCopyByLoad**(e_L, $\mathcal{O}$), and since $o \in \mathcal{O}$ (because $o \in pts(q)$), e_C and e_L will be bound, and if e_C does not already exist, Algorithm 2 will add e_C to the graph, thus the assumption is invalid.

— *Store Constraint*: $\forall p \xleftarrow{\text{store}} q \in \mathcal{E}$, if $o \in pts(p)$, then there must be a CopyEdge $o \xleftarrow{\text{copy}} q \in \mathcal{E}$. Suppose there is a StoreEdge $e_S = p \xleftarrow{\text{store}} q$ and an object $o \in pts(p)$, such that $e_C = o \xleftarrow{\text{copy}} q \notin \mathcal{E}$. Then there are two possible cases: (a) o was inserted into $pts(p)$ due to a certain change, and Algorithm 2 fails to solve the Load constraint related to p based on the change in $pts(p)$; (b) e_S is newly added to the constraint graph, and Algorithm 2 fails to solve this new Load constraint. For cases (a), when Algorithm 2 updates $pts(p)$ in Part 3, according to the rule **FilterInsertion**, o must be in Δ_{diff} at this point, otherwise o would not have been newly inserted into $pts(p)$. Since e_S is a LoadEdge from q to p, according to **PropagateInsertion**(p, Δ_{diff}), we have $q \in \mathcal{Y}$, and we also have **InsCopyByLoad**(e_L, Δ_{diff}). Then, based on the rules **InsCopyByStore**, **Bind**, and **NewCopyEdge**, since $o \in \Delta_{diff}$ (i.e., $o \in \mathcal{O}$), e_C and e_S will be bound, and if e_C does not already exist, Algorithm 2 will add e_C to the graph, thus the assumption is invalid. For case (b), when Algorithm 2 inserts edges in Part 1, according to the rule **InsStore**, we have **InsCopyByStore**(e_S, $\mathcal{O}$), and since $o \in \mathcal{O}$ (because $o \in pts(p)$), e_C and e_S will be bound, and if e_C does not already exist, Algorithm 2 will add e_C to the graph, thus the assumption is invalid.

— *Copy Constraint*: $\forall p \xleftarrow{\text{copy}} q \in \mathcal{E}, pts(p) \subseteq pts(q)$, that is, $\forall o \in pts(q)$, o must be in $pts(p)$. Suppose there is a CopyEdge $e_C = p \xleftarrow{\text{copy}} q$, and an object $o \in pts(q)$, such that $o \notin pts(p)$. Then there are two possible cases: (a) o is inserted into $pts(q)$ due to a certain change, and Algorithm 2 fails to propagate the changes in $pts(q)$ to $pts(p)$; (b) e_C is newly added to the constraint graph, and Algorithm 2 fails to solve this new Copy constraint. For case (a), when Algorithm 2 updates $pts(q)$ in Part 3, according to the rule **FilterInsertion**, o must be in Δ_{diff} at this point, otherwise o would not have been newly inserted into $pts(q)$. Since e_C is a CopyEdge from q to p, according to **PropagateInsertion**(q, Δ_{diff}), we have $p \in \mathcal{W}$, and Algorithm 2 will merge Δ_{diff} into $\Delta(p)$. Then, when Algorithm 2 processes p, since $o \in \Delta(w)$, according to the rules **FilterInsertion** and **PropagateInsertion**, o will ultimately be in $pts(p)$, thus the assumption is invalid. For scenario (b), since e_C is newly added to the constraint graph and $o \in pts(q)$, according to the rule **NewCopyEdge**, Algorithm 2 will merge $pts(q)$ into $\Delta(p)$. Finally, according to the rules **FilterInsertion** and **PropagateInsertion**, o will ultimately be in $pts(p)$, thus the assumption is invalid.

Algorithm 1: Incremental Pointer Constraint Edge Deletion

Input: A worklist of constraint edges to be deleted: $WL : \{e_1, e_2, ...\}$.
The constraint graph: $\mathcal{G} = \{\mathcal{N}, \mathcal{E}, pts\}$.

```
1  R ← ∅;
2  Δ ← ∅;
   // Part 1: Constraint Edge Deletion
3  while WL ≠ ∅ do
4      e ← WL.Pop();
       // Apply rules in Figure 6 by edge types.
5      DelEdge(e);
6  ReAnalyze ← TRUE;
7  while ReAnalyze do
8      ReAnalyze ← FALSE;
       // Part 2: SCC Broken Detection
9      while R ≠ ∅ do
10         r, E_r ← PopKeyAndValue(R);
11         SCCBrokenDetection(r, E_r);
       // Part 3: Points-To Set Updating and Propagation
12     N_T = GetTopoOrder(G);
13     while N_T ≠ ∅ do
14         p ← N_T.Pop();
15         if Δ(p) ≠ ∅ then
16             FilterAndPropagateDel(p, Δ(p));
17     Δ ← ∅;
18     UpdateIndirectCallEdge(P_func);
19     if R ≠ ∅ or Δ ≠ ∅ then
20         ReAnalyze ← TRUE;
```

4.2.6 Incremental PTA: End-to-End Workflow. As with other recent work on incremental PTA [31–33], we divide program changes into two categories: statement deletion and insertion (a modification of program statements can be treated as a combination of deletion and insertion). Therefore, our end-to-end incremental PTA takes as input the set of deletion statements and the set of insertion statements from the program. We follow the approach of existing work [32, 33] to extract the corresponding constraint edges that need to be modified based on the changed statements. Specifically, for each changed statement, we first extract the corresponding constraint edges (i.e., for deleted statements, we extract edges from the constraint graph, and for new statements, we create new constraint edges) according to Table 1 and push them into the worklists.[4] We then call Algorithm 1 on the deleted-edge-worklist and Algorithm 2 on the inserted-edge-worklist to compute and propagate changes of the points-to information and update the constraint graph.

We adopt the practice of deleting before inserting, as this is a common approach in existing incremental analyses [32, 33]. However, we have also discovered that strategically arranging the order of processing various change statements can reduce the algorithm's redundant computations. For instance, in certain scenarios, processing additions before deletions can prevent the need to

[4] Counters are used to record the correspondence between each constraint edge and program statements, addressing scenarios where multiple statements correspond to a single edge (e.g., when multiple *p = q statements exist in the program, and only one (some) of them is (are) deleted, the corresponding StoreEdge should not be extracted to remove).

Algorithm 2: Incremental Pointer Constraint Edge Insertion

Input: A worklist of constraint edges to be inserted: $WL : \{e_1, e_2, ...\}$.
The constraint graph: $\mathcal{G} = \{\mathcal{N}, \mathcal{E}, pts\}$.

```
1  E_new ← ∅ ;
2  Δ ← ∅;
   // Part 1: Constraint Edge Insertion
3  while WL ≠ ∅ do
4      e ← WL.Pop();
       // Apply rules in Figure 10 by edge types.
5      InsEdge(e);
6  ReAnalyze ← TRUE;
7  while ReAnalyze do
8      ReAnalyze ← FALSE;
       // Part 2: SCC Cycle Detection
9      if E_new ≠ ∅ then
10         SCCDetect(E_new);
11         E_new ← ∅ ;
       // Part 3: Points-To Set Updating and Propagation
12     N_T = GetTopoOrder(G);
13     while N_T ≠ ∅ do
14         p ← N_T.Pop();
15         if Δ(p) ≠ ∅ then
16             FilterAndPropagateIns(p, Δ(p));
17     Δ ← ∅;
18     UpdateIndirectCallEdge(P_func);
19     if E_new ≠ ∅ or Δ ≠ ∅ then
20         ReAnalyze ← TRUE;
```

remove certain CopyEdges from the constraint graph by managing the size of their binding sets to increase from 1 to 2 and then decrease back to 1, instead of decreasing from 1 to 0, which would necessitate removing the CopyEdge if deletions were processed first. This optimization is beyond the scope of this article. We view it as future work and believe it presents an interesting and effective avenue for further enhancing the performance of incremental algorithms.

Theorem 4.3 (Correctness of Incremental PTA). *Our incremental PTA is correct because, for two programs P_{old} and P_{new} before and after the change, and for all deleted edges C_D and inserted edges C_I, applying our incremental PTA to C_I on the full analysis results $Res(P_{\text{old}})$ yields analysis results $IncRes_I(Res(P_{\text{old}}), C_D, C_I)$ that are consistent with the full analysis results $Res(P_{\text{new}})$ of the program after the change.*

Proof. Let P_{old} become P' after applying the deletion changes C_D, and P' become P_{new} after applying the insertion changes C_I. According to Theorem 4.1, the results $IncRes_D(Res(P_{\text{old}}), C_D)$ are consistent with $Res(P')$, and according to Theorem 4.2, $IncRes_I(Res(P'), C_I)$ are consistent with $Res(P_{\text{new}})$. Naturally, $IncRes(Res(P_{\text{old}}), C_D, C_I)$ is consistent with $Res(P_{\text{new}})$. □

$$\textbf{InterSummary}\ \frac{\begin{array}{c}cs = [_ = \mathcal{F}_{cs}(_)] \text{ is a call site in function } g \\ f \in Callees(\mathcal{F}_{cs}) \\ \mathcal{S}(f) \text{ is the summary of } f\end{array}}{\mathcal{S}_{\text{inter}}(g) = \mathcal{S}_{\text{inter}}(g) \cup \textbf{op}(\mathcal{S}(f))} \qquad \textbf{IntraSummary}\ \frac{\begin{array}{c}s \text{ is a non-call statement in function } g \\ \mathcal{S}(s) \text{ is the summary of } s\end{array}}{\mathcal{S}_{\text{intra}}(g) = \mathcal{S}_{\text{intra}}(g) \cup \mathcal{S}(s)}$$

$$\textbf{OverallSummary}\ \frac{\begin{array}{c}g \text{ is a function} \\ \mathcal{S}_{\text{inter}}(g) \text{ is the inter-summary of } g \\ \mathcal{S}_{\text{intra}}(g) \text{ is the intra-summary of } g\end{array}}{\mathcal{S}(g) = \mathcal{S}_{\text{inter}}(g) \cup \mathcal{S}_{\text{intra}}(g)}$$

Fig. 12. The inference rules of FSA.

4.3 Incremental MRA

In this section, we introduce our incremental algorithm suitable for other bottleneck module of SVFA—MRA. As previously introduced (Section 2.2), MRA is an instance of FSA. We claim that our incremental algorithm framework is applicable not only to MRA but also to all function summary analyses similar to MRA.

Existing methods [2, 49, 56, 57, 66, 80] for incrementally computing function summaries generally rely on the idea of reset-recompute, which can be traced back to the **deletion-rederivation (DRed)** algorithm [16]. Unlike these methods, our approach only propagates the changed elements within the summary, instead of resetting and recomputing the summaries of all functions reachable by changes (as discussed in Section 3.2). Another key insight of our approach is that function summaries can be categorized into inter-procedural and intra-procedural summaries, depending on whether they are derived from call statements or non-call statements. When updating incremental summaries, if an element is removed from a function's inter-procedural summary but remains in the intra-procedural summary, the overall summary still retains the element. Since the element in the overall summary remains unchanged, we can prevent it from propagating to the function's callers, thereby minimizing the number and narrowing the scope of propagated summary elements.

Next, we first introduce our incremental approach for FSA (Section 4.3.1), and then show how to utilize such an approach for incremental MRA (Section 4.3.2).

4.3.1 Incremental FSA: Algorithm. Figure 12 shows the inference rules for FSA whose summary are categorized into inter- and intra-procedural types. The summary of a function $\mathcal{S}(g)$ is determined by all the statements within the function g. Since there may be call statements within a function, the summary of a callee can also affect the summary of its caller through the call relationship. We make a distinction between function summaries defined by call statements and those defined by other statements, labeling them as inter-summary ($\mathcal{S}_{\text{inter}}$) and intra-summary ($\mathcal{S}_{\text{intra}}$), respectively. Next, we explain the rules shown in Figure 12.

Rule **InterSummary** shows the computation of the summary determined by call sites. If $cs = [_ = \mathcal{F}_{cs}(_)]$ is a call site in function g and f is one of its callees (i.e., $f \in Callees(\mathcal{F}_{cs})$), then the summary of the callee (i.e., $\mathcal{S}(f)$), is merged into the caller's inter-summary $\mathcal{S}_{\text{inter}}(g)$. In general, passing the summary between callee and caller is affected by the actual parameters and return values at the call site, so we need a specific operation, $\textbf{op}(.)$, defined by the instantiation analysis (e.g., the $(. \cap E(cs))$ operation in Call rule of MRA discussed in Table 2 or Section 4.3.2), to filter the callee's summary which would be merged into the caller's summary. Rule **IntraSummary** is applied to compute the summary determined by non-call statements. For each non-call statement s in function g, its summary $\mathcal{S}(s)$ is directly merged into $\mathcal{S}_{\text{intra}}(g)$. Finally, according to rule **OverallSummary**, the inter-summary $\mathcal{S}_{\text{inter}}$ and the intra-summary $\mathcal{S}_{\text{intra}}$ are merged to construct the overall summary $\mathcal{S}$ of the function.

Algorithm 3: Incremental FSA

```
   Input: The old call graph: CG_old.
          The new call graph: CG_new.
1  F_I ← ∅;
2  foreach f ∈ CG_old ∪ CG_new do
       // Record f's intra-summary before updating.
3      S_intra_old(f) ← S_intra(f);
4      S_intra(f) ← ComputeNewIntraSummary(f);
5      if S_intra_old(f) ≠ S_intra(f) then
           // Record f if its intra-summary changed
6          F_I.insert(f);
7  ClearSummaryElement(F_I, CG_old, CG_new);
8  ReAddSummaryElement(CG_new);
```

Now, we present our incremental approach for FSA in Algorithm 3. For simplicity, we treat various function summaries as globally accessible data structures rather than explicit inputs to Algorithm 3 (the same applies to Algorithms 4 and 5). The inputs of the algorithm are the old and new call graphs. Firstly, functions are collected into $\mathcal{F}_I$ if their intra-summaries have changed (lines 2 to 6). Specifically, we apply the rule **IntraSummary** to compute the new intra-summary for all functions. Note that, since the computation of intra-summaries is linear in complexity, this step does not incur significant time overhead. In particular, if a function $f \notin CG_{\text{old}}$, its old intra-summary is $\emptyset$. Similarly, if a function $f \notin CG_{\text{new}}$, its new intra-summary is $\emptyset$. The subsequent two steps constitute the core of the algorithm: (1) clearing the deleted summary elements along the call edges of the old call graph (line 7); (2) re-adding the inter-summary on the new call graph (line 8).

Algorithm 4 gives the details of clearing functions' summary elements. Given a set of functions whose intra-summary is changed, for each of them, the algorithm updates its overall summary by merging its new intra-summary while removing elements that only occurred in its old intra-summary (lines 1 to 7). Specifically, if a function is deleted, we no longer update its overall summary and instead set it to empty (line 5). Next, we sort the functions in reverse topological order based on the call relationship in CG_{old} and store them in WL (line 8), in order to analyze functions in a bottom-up fashion. It is important to note that during the sorting, cycles may exist in the call graph due to recursion; therefore, we first compress SCCs in the call graph and then perform a topological sort on the resulting acyclic graph. Nodes within the same SCC share the same topological order. The reverse topological order minimizes redundant calculations because the inter-summary is passed from the bottom up along the call graph. Subsequently, we continuously take a function from WL and pass its summary deletion $\mathcal{S}_\Delta$ to its callers. There are two types of summary deletion: (1) the removal of a call edge requires the deletion of elements from the callee's summary in the caller's summary (line 15), and (2) elements deleted from the callee's summary must be propagated along unchanged call edges to be deleted from the callee's summary (line 17). After computing the summary deletion (i.e., $\mathcal{S}_\Delta$) for function f, we propagate it to the callers of f following the procedure `PropSummaryDeletion`. `PropSummaryDeletion` is employed to clear the reduction of the summary elements, $\mathcal{S}_\Delta$, in caller g's summary. The intuition is that for any element in $\mathcal{S}(g)$, if it was produced solely by g's callees and not by g's non-call statements, and it has now been deleted from the summary of one of g's callees, then we remove o from $\mathcal{S}(g)$ (lines 23 to 26). Additionally, we mark this situation (line 26) and put the caller g in the WL (line 20). Finally, we remove all the elements in $\mathcal{S}_\Delta$ from the inter-summary of g (line 27).

Algorithm 4: Clear Summary Elements

```
Input: Set of intra-summary changed functions: F_I.
       The old call graph: CG_old.
       The new call graph: CG_new.
1  foreach f ∈ F_I do                                   // Update f's overall summary
2      S_old(f) ← S(f) ;                  // Record overall summary of f before updating.
3      S_D ← S_intra_old(f)\S_intra(f) ;                // S_intra(f) = ∅ when f is deleted.
4      if f ∉ CG_new then
5          S(f) ← ∅ ;          // Set the deleted function's overall summary to empty.
6      else
7          S(f) ← S(f) ∪ S_intra(f)\S_D;
8  WL ← getCGRevTopoOrder(CG_old);
9  while WL ≠ ∅ do
10     f ← WL.Pop();
11     foreach g ∈ Callers(f) by CG_old do
12         Change ← FALSE;
13         S_Δ ← ∅;
14         if g ∉ Callers(f) by CG_new then
15             S_Δ ← op(S_old(f));
16         else
17             S_Δ ← op(S_old(f))\op(S(f));
18         Change ← PropSummaryDeletion(g, S_Δ);
19         if Change = TRUE then
20             WL.push(g);
21 Procedure PropSummaryDeletion(g, S_Δ)
22     Change ← FALSE;
23     foreach o ∈ S_Δ do
24         if o ∈ S_inter(g) ∧ o ∉ S_intra(g) then
25             S(g) ← S(g)\{o};
26             Change ← TRUE;
27     S_inter(g) ← S_inter(g)\S_Δ;
28     return Change
```

Algorithm 5 describes the process of re-adding the summary elements. Similarly, the algorithm processes each function bottom-up on the call graph (line 1). Then we compute the inter-summary and overall summary to guarantee that both are consistent with the full analysis. Technically, for each function f in the worklist, we first apply **op**(.) to filter task-specific summary information, then merge the filtered summary with $\mathcal{S}_{\text{inter}}(g)$ (respectively, $\mathcal{S}(g)$), the inter-summary (respectively, overall

Algorithm 5: Re-Add Summary Elements

```
Input: Call graph of new program: CG_new
1  WL ← getCGRevTopoOrder(CG_new);
2  while WL ≠ ∅ do
3      f ← WL.Pop();
4      foreach g ∈ Callers(f) do
5          S_inter(g) ← S_inter(g) ∪ op(S(f));
6          S(g) ← S(g) ∪ op(S(f));
7          if S(g) is changed then
8              WL.push(g);
```

summary) of f's caller g (lines 5 and 6). If the caller's overall summary changes, the caller is placed in the *WL* for further iteration.

Now we prove the correctness of our incremental FSA algorithm.

Theorem 4.4 (Correctness of Incremental FSA Algorithm). *Our incremental FSA algorithm is correct because, for any function g in the new version of the program, the algorithm ensures that the final result satisfies $\mathcal{S}(g) = \mathcal{S}_{\text{intra}}(g) \cup \mathcal{S}_{\text{inter}}(g)$, which means that the incremental analysis results are consistent with those from a full analysis.*

Proof. We prove that the result of the algorithm always satisfies $\mathcal{S}(g) = \mathcal{S}_{\text{intra}}(g) \cup \mathcal{S}_{\text{inter}}(g)$.

- We first prove $\mathcal{S}(g) \supseteq \mathcal{S}_{\text{intra}}(g) \cup \mathcal{S}_{\text{inter}}(g)$ by dividing it into two cases:
  Case 1: $\forall o \in \mathcal{S}_{\text{intra}}(g)$, o must be in $\mathcal{S}(g)$. Because (1) $\mathcal{S}_{\text{intra}}(g)$ is incorporated into $\mathcal{S}(g)$ (line 7), and (2) o is never removed from $\mathcal{S}(g)$ if $o \in \mathcal{S}_{\text{intra}}(g)$ (line 24) according to Algorithm 4.
  Case 2: $\forall o \in \mathcal{S}_{\text{inter}}(g)$, o must be in $\mathcal{S}(g)$. Because (1) $\mathcal{S}_{\text{inter}}(g) = \bigcup_{f \in Callees(g)} \mathbf{op}(\mathcal{S}(f))$ (line 5), and (2) $\mathbf{op}(\mathcal{S}(f))$ is incorporated into $\mathcal{S}(g)$ (line 6) according to Algorithm 5.
  In summary, $\mathcal{S}(g) \supseteq \mathcal{S}_{\text{intra}}(g) \cup \mathcal{S}_{\text{inter}}(g)$.
- We then prove $\mathcal{S}(g) \subseteq \mathcal{S}_{\text{intra}}(g) \cup \mathcal{S}_{\text{inter}}(g)$ by contradiction:
  Suppose $\mathcal{S}(g) \nsubseteq \mathcal{S}_{\text{intra}}(g) \cup \mathcal{S}_{\text{inter}}(g)$, that is, $\exists o \in \mathcal{S}(g)$ *s.t.* $o \notin \mathcal{S}_{\text{intra}}(g) \wedge o \notin \mathcal{S}_{\text{inter}}(g)$. According to line 7 in Algorithm 4 and line 6 in Algorithm 5, the elements inserted into $\mathcal{S}(g)$ by the algorithm can only originate from $\mathcal{S}_{\text{intra}}(g)$ or $\mathcal{S}_{\text{inter}}(g)$. So o cannot be a newly inserted element. It must have been in $\mathcal{S}(g)$ before the execution of the incremental algorithm. This implies that o was either in the old intra-summary old intra-summary $\mathcal{S}_{\text{intra_old}}(g)$ or old inter-summary $\mathcal{S}_{\text{inter_old}}(g)$, because the summary before the incremental analysis satisfies $\mathcal{S}_{\text{old}}(g) = \mathcal{S}_{\text{intra_old}}(g) \cup \mathcal{S}_{\text{inter_old}}(g)$.
  Case 1: $o \in \mathcal{S}_{\text{intra_old}}(g)$, since $o \notin \mathcal{S}_{\text{intra}}(g)$, o must be in $\mathcal{S}_D$ (line 3 in Algorithm 4) and it must be removed from $\mathcal{S}(g)$ (line 7 in Algorithm 4). So $o \notin \mathcal{S}(g)$, which contradicts the supposition.
  Case 2: $o \in \mathcal{S}_{\text{inter_old}}(g)$, since $o \notin \mathcal{S}_{\text{inter}}(g)$, o must be in $\mathcal{S}_\Delta$ (line 17 in Algorithm 4) and it must be removed from $\mathcal{S}(g)$ (line 25 in Algorithm 4). So $o \notin \mathcal{S}(g)$, which contradicts the supposition. In Summary, there is no such $o \in \mathcal{S}(g)$ s.t. $o \notin \mathcal{S}_{\text{intra}}(g) \wedge o \notin \mathcal{S}_{\text{inter}}(g)$. The hypothesis is not valid, so $\mathcal{S}(g) \subseteq \mathcal{S}_{\text{intra}}(g) \cup \mathcal{S}_{\text{inter}}(g)$. □

4.3.2 Incremental MRA. In this section, we introduce how to use the incremental function-summary-based analysis framework described earlier for incremental MRA.

Figure 13 shows how the MRA is treated as an FSA. We naturally distinguish the Mod/Ref set computed by the store/load statement (according to rule for Store and Load in Table 2) from those computed by the call statement (according to rule for Call in Table 2) and treat them as intra-Mod/Ref set ($\mathcal{D}_{\text{intra}}/\mathcal{U}_{\text{intra}}$) and inter-Mod/Ref set ($\mathcal{D}_{\text{inter}}/\mathcal{U}_{\text{inter}}$), respectively. The specific operation corresponding to **op**(.) in rule **InterSummary** is $(. \cap E(cs))$ in rules **InterMod** and **InterRef**. Finally, the overall Mod/Ref set (i.e., the Mod/Ref set of a function) obtained by combining intra- and inter-Mod/Ref sets is equivalent to that computed by combining all statements' mod/ref sets according to rule for Procedure in Table 2.

Recall that we compute the new intra-summaries for all functions in our incremental FSA algorithm and collect the functions whose intra-summary changed into $\mathcal{F}_I$. For incremental MRA, we can use the results of incremental PTA to improve the efficiency of this step: Only the intra-Mod/Ref sets of the function which contains the changed load/store statement[5] or the

[5]If a store/load statement is deleted/inserted, or the points-to set of its dereferenced pointer is changed, we call it a changed store/load statement.

$$\textbf{InterMod}\ \frac{\begin{array}{c}cs = [_ = \mathcal{F}_{cs}(_)] \text{ is a call site in function } g \\ f \in Callees(\mathcal{F}_{cs}) \\ \mathcal{D}(f) \text{ is the Mod set of } f\end{array}}{\mathcal{D}_{\text{inter}}(g) = \mathcal{D}_{\text{inter}}(g) \cup (\mathcal{D}(f) \cap E(cs))} \qquad \textbf{IntraMod}\ \frac{\begin{array}{c}s = [*p = q] \text{ is a store in function } g \\ \mathcal{D}(s) = \{o \mid o \in pts(p) \wedge o \in NonLocal_g\}\end{array}}{\mathcal{D}_{\text{intra}}(g) = \mathcal{D}_{\text{intra}}(g) \cup \mathcal{D}(s)}$$

$$\textbf{OverallMod}\ \frac{\begin{array}{c}g \text{ is a function} \\ \mathcal{D}_{\text{inter}}(g) \text{ is the inter-Mod set of } g \\ \mathcal{D}_{\text{intra}}(g) \text{ is the intra-Mod set of } g\end{array}}{\mathcal{D}(g) = \mathcal{D}_{\text{inter}}(g) \cup \mathcal{D}_{\text{intra}}(g)}$$

$$\textbf{InterRef}\ \frac{\begin{array}{c}cs = [_ = \mathcal{F}_{cs}(_)] \text{ is a call site in function } g \\ f \in Callees(\mathcal{F}_{cs}) \\ \mathcal{U}(f) \text{ is the Ref set of } f\end{array}}{\mathcal{U}_{\text{inter}}(g) = \mathcal{U}_{\text{inter}}(g) \cup (\mathcal{U}(f) \cap E(cs))} \qquad \textbf{IntraRef}\ \frac{\begin{array}{c}s = [p = *q] \text{ is a load in function } g \\ \mathcal{U}(s) = \{o \mid o \in pts(q) \wedge o \in NonLocal_g\}\end{array}}{\mathcal{U}_{\text{intra}}(g) = \mathcal{U}_{\text{intra}}(g) \cup \mathcal{U}(s)}$$

$$\textbf{OverallRef}\ \frac{\begin{array}{c}g \text{ is a function} \\ \mathcal{U}_{\text{inter}}(g) \text{ is the inter-Ref set of } g \\ \mathcal{U}_{\text{intra}}(g) \text{ is the intra-Ref set of } g\end{array}}{\mathcal{U}(g) = \mathcal{U}_{\text{inter}}(g) \cup \mathcal{U}_{\text{intra}}(g)}$$

Fig. 13. The inference rules of incremental MRA.

points-to-set-changed pointers need to be recomputed, because only the intra-Mod/Ref set of these functions may change.

By simply treating the Mod/Ref set as a function summary in accordance with the rules in Figure 13, we can effortlessly utilize the incremental FSA framework (Algorithms 4 and 5) to incrementally update the function's Mod/Ref set.

Lemma 4.5 (Correctness of Incremental MRA). *Our incremental MRA is correct, that is, the incrementally updated Mod/Ref sets of each function by our incremental MRA are consistent with those obtained through the full MRA.*

Proof. MRA is an instance of FSA. Since Theorem 4.4 ensures the correctness of the incremental FSA, meaning that the analysis results of the incremental FSA are consistent with those of the full FSA, it naturally follows that the analysis results of the incremental MRA are also consistent with those of the full MRA. □

5 Evaluation

We implemented SILVA based on SVF[6] [60], a state-of-the-art open source static SVFA. Our evaluation aims to answer the following **research questions (RQs)**:

RQ1: How efficient is SILVA in analyzing real-world programs compared to SVF?
RQ2: Are our two incremental algorithms more efficient than existing incremental algorithms?
RQ3: How does the performance of SILVA vary with code changes at different scales?
RQ4: Are the analysis results of SILVA consistent with the results of a full analysis?

[6]We used SVF-2.7 (https://github.com/SVF-tools/SVF/releases/tag/SVF-2.7), which was the latest version available when we began our implementation. It supports up to LLVM-14 [29].

Table 3. Benchmarks and the Program Characteristics

Project	#LoC	#Pointer	#Object	#Function	#CallSite
Tmux	68.0K	272.0K	15.2K	2.5K	12.8K
Git	287.3K	1.0M	91.1K	12.3K	70.7K
Vim	432.7K	995.1K	58.8K	7.7K	54.9K
OpenSSL	589.3K	838.3K	76.8K	13.3K	58.2K
PostgreSQL	1.0M	2.5M	191.5K	24.2K	180.6K

#Pointer, #Object, #Function, and #CallSite represent the average number of pointers, objects, functions, and call sites across all commits of the analyzed program.

To answer *RQ1*, we run both SVF and SILVA in real-world code commit scenarios and evaluate the time required for the two tools (Section 5.1). To address *RQ2*, we conduct comparative experiments separately for the PTA and MRA modules: We implement the IPA algorithm [33] and the classic reset-recompute algorithm [16] for the PTA module and the MRA module in our framework, respectively. We then compare these with our incremental algorithms using real-world code commits (Section 5.2). To answer *RQ3*, we use code changes of different sizes for each project to evaluate SILVA (Section 5.3). To address *RQ4*, we compared the analysis results of SVF and SILVA across all experiments. This comparison helps validate the correctness of SILVA's implementation. We also compare the analysis results of our reproduced existing incremental algorithms (i.e., IPA and the classic reset-recompute algorithm) with the results of the full analysis to ensure their correctness. All our experiments are conducted on a computer with Intel(R) Xeon(R) CPU E5-2660 v4 @ 2.00 GHz and 256 GB RAM running Ubuntu-22.04.

Benchmarks. We select five C/C++ projects (Table 3) as benchmarks. All of them are widely used and actively maintained open source real-world projects. Notably, many recent static analyses for C/C++ [4, 34, 52] have used these programs in their evaluation. In this evaluation, we focus on code changes between commits from the projects' GitHub repositories. Here, the code changes refer to specifically the edits in C/C++ source code, and we ignore those made in non-C/C++ files (e.g., build files, text files, or comments in C/C++ files).

5.1 The Performance of SILVA in Analyzing Real-World Code Changes

We evaluate the performance of SILVA on real-world programs using code changes from code commits. Specifically, we first select code commits from the target project's GitHub repository. For each commit, we extract the change between it and its preceding commit. Finally, we assess the performance of SILVA by running it on the changes extracted from the commit. We design two types of code commits to understand the efficiency and adaptability of SILVA. Firstly, we select consecutive code commits to assess the time savings of SILVA in a continuous development setting (Section 5.1.1). This evaluation aims to measure SILVA's ability to efficiently analyze incremental changes that are common in a rapid development cycle. Secondly, we conduct experiments on randomly selected non-consecutive commits to evaluate the generality of SILVA with respect to diverse code changes (Section 5.1.2). Since non-consecutive commits are likely to involve more varied changes across different parts of a large project, this approach helps in assessing whether SILVA can effectively adapt to and analyze a broader range of modifications, thus providing a more comprehensive evaluation of its performance across different scenarios in a target project.

5.1.1 *The Performance of SILVA in Analyzing Consecutive Code Commits.*

Experiment Setup. We conduct experiments on consecutive code commits using a method similar to

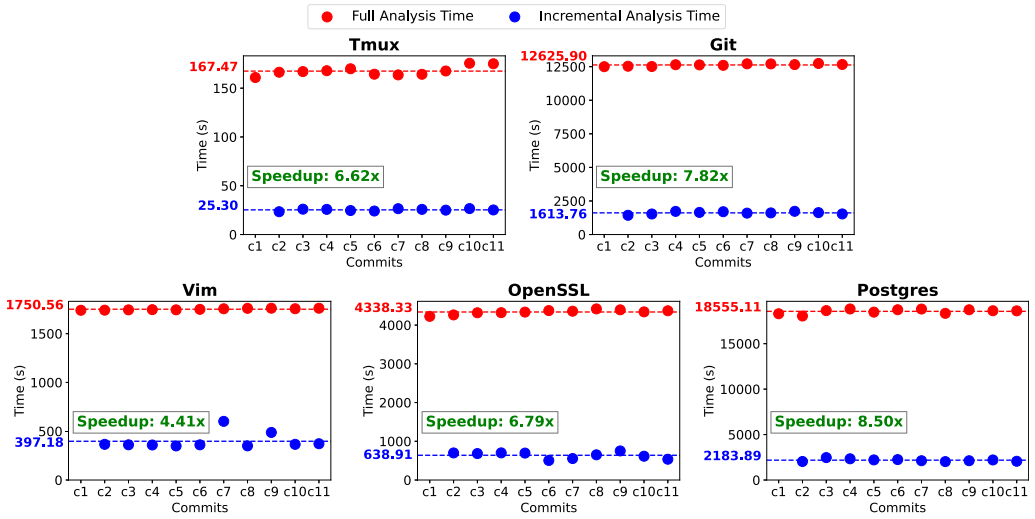


Fig. 14. Performance of SILVA comparing with SVF when running on consecutive commits. The *x*-axis represents each commit, where "c1" is the earliest commit version initialized with full analysis, and the *y*-axis represents the analysis time. The speed-up factor of incremental SVFA compared to full SVFA is marked in green font. The red dashed line represents the average time for the full SVFA, while the blue dashed line represents the average time for the incremental SVFA.

those employed in other incremental studies [2, 32]. Specifically, from all the code commit versions between the two latest stable versions in each project's GitHub repository, we select 11 consecutive valid commit versions (i.e., we skip and bypass those commit versions that do not contain C/C++ code changes). For each project, we first conduct a full SVFA on the earliest commit version and use the analysis results (i.e., the constraint graph, call graph, and the Mod/Ref set of each function) to initialize SILVA. Then, for each subsequent commit version, we sequentially compute the code changes between that version and the previous one. We use these changes as the input for SILVA to run incremental SVFA each time, and we record the time spent. Also, for each commit version of every project, we run SVF to record the time taken for the full SVFA.

Results. Figure 14 shows the detailed time performance comparison between SILVA and SVF when analyzing consecutive commit versions. The green font indicates the average speedup of the incremental analysis over the full analysis across all commits.

Overall, SILVA demonstrates significant speed improvements across all target projects. For Vim, which has a higher average number of lines changed, the acceleration was relatively lower at 4.41× faster. Conversely, for PostgreSQL, which is larger and has fewer lines of code changed per update, the improvement is most pronounced, reaching up to 8.5× faster.

For most projects, the incremental analysis time of SILVA was relatively stable. However, for Vim, the incremental analysis time experienced fluctuations (for example, the time reached about 600 seconds for commit c7 in Figure 14). This variability was due to the larger scale of code changes in that commit compared to the previous one, totaling over 250 lines.[7]

5.1.2 *The Performance of SILVA in Analyzing Non-Consecutive Code Commits.*

Experimental Setup. We randomly select 10 non-consecutive commits from the GitHub repository of the target projects over the past 2 years and evaluate SILVA according to the previously described

[7] https://github.com/vim/vim/commit/f93b1c881a99fa847a1bafa71877d7e16f18e6ef.

Table 4. Performance of SILVA Comparing with SVF When Running on Code Commits

Project	#Stmt	Full VFA (s)	Inc VFA (s)	Full PTA+MRA (s)	Inc PTA+MRA (s)
Tmux	+68.8/−12.6	171.37	24.58 (6.97×)	158.38	6.93 (22.85×)
Git	+15.0/−10.5	12,648.09	1,595.25 (7.93×)	11,982.63	925.68 (12.94×)
Vim	+92.9/−14.3	1,761.94	443.78 (3.97×)	1,508.17	186.04 (8.11×)
OpenSSL	+28.5/−12.9	4,183.92	739.53 (5.66×)	3,877.05	429.37 (9.03×)
PostgreSQL	+16.5/−12.7	18,204.92	2,476.79 (7.35×)	16,815.06	1,081.94 (15.54×)
Avg.	+44.3/−12.6	7,394.05	1,055.99 (7.00×)	6,868.26	525.99 (13.06×)

The "**#Stmt**" column presents the average number of added and deleted statements per commit. The "**Full VFA**" indicates the time spent on the full sparse value-flow analysis by SVF, and "**Inc VFA**" indicates the time spent on the incremental sparse value-flow analysis by SILVA. The "**Full PTA+MRA**" indicates the time spent on the full pointer analysis and full Mod-Ref analysis by SVF, and "**Inc PTA+MRA**" indicates the time spent on the incremental pointer analysis and incremental Mod-Ref analysis by SILVA. The red font highlights the acceleration of the incremental analysis in SILVA compared to the full analysis in SVF.

experimental steps. Specifically, we perform a full SVFA using the code version before each commit to initialize SILVA,[8] then use the changes from the commit as input to run incremental SVFA. We run each tool on each commit and record the average time spent on VFA as well as the average time spent on the two bottleneck modules (i.e., PTA and MRA).

Results. Table 4 lists the detailed data in the performance comparison between SILVA and SVF when analyzing non-consecutive commits. Column 2 presents the average number of added and deleted statements (denoted + and −) for the 10 git commits for each project. Columns 3 and 5 give the times of SVF's full analyses, while columns 4 and 6 are the times of SILVA's incremental analyses.

On average, SILVA requires only 1,055.99 seconds to complete an incremental SVFA for a single commit, which is 7× faster than the full analysis time of 7,396.05 seconds. In other words, the SVFA that previously required over 2 hours can now be completed by SILVA in less than 20 minutes. A general trend observed is that for projects with larger scales, where full analysis takes more time and code changes are relatively fewer, such as Git (although not extensive in code lines, it has a significantly large number of pointers, objects, functions, and call sites as shown in Table 3) and PostgreSQL, the acceleration multiplier of SILVA becomes more pronounced. When examining the data for the two bottleneck modules, we observe that SILVA achieves a significantly greater speedup compared to SVF, with an average improvement of over 13×. This further demonstrates the effectiveness of our incremental algorithms.

In summary, in real-world consecutive code commit scenarios, SILVA demonstrates a significant speed-up compared to the state-of-the-art full layered SVFA SVF.

5.2 Comparative Experiments of the Two Bottleneck Modules

Experimental Setup. To better understand the acceleration effects of our two incremental algorithms on the two bottleneck modules, we conduct comparative evaluations. We implement existing incremental algorithms for comparison in our experiments: For the PTA module, we compare the existing state-of-the-art Andersen-style incremental PTA algorithm, IPA [33], with our incremental PTA algorithm, both of which are context- and flow-insensitive. Note that in our implementation, we supplement IPA with multi-source derivation edge binding and unbinding rules (Section 4.2.3) to ensure that IPA can correctly handle the deletion/insertion of derived CopyEdges during incremental analysis, just like our algorithm. For the MRA module, we compare the classic reset-recompute

[8]For example, if we randomly select non-consecutive commits such as c3, c6, and c9, we would use commits c2, c5, and c8, respectively, to run full analysis to initialize SILVA.

Table 5. Statistics of Different Pointer Analyses under Each Commit Interval

Project	i	Full	IPA			Ours		
		Time (s)	Time (s)	#SCC	#Node	Time (s)	#SCC	#Node
Tmux	1	128.49	4.70 (27.33×)	15.4	53.4	0.65 (197.68×)	1.5 (↓90.25%)	31.4 (↓41.20%)
	10	132.26	74.84 (1.77×)	195.2	563.6	1.47 (89.97×)	5.4 (↓97.23%)	446.7 (↓20.74%)
	20	130.05	170.83 (0.76×)	932.3	1,216.9	2.27 (57.29×)	8.1 (↓99.13%)	980.5 (↓19.43%)
	30	132.37	237.48 (0.56×)	1,209.1	1,890.0	3.28 (40.36×)	12.3 (↓98.98%)	1,250.3 (↓33.85%)
Git	1	1,396.15	61.05 (22.87×)	45.4	54.7	2.75 (507.69×)	1.3 (↓97.14%)	22.3 (↓59.23%)
	10	1,417.58	95.43 (14.85×)	66.0	96.1	6.71 (211.26×)	3.4 (↓94.85%)	68.6 (↓28.62%)
	20	1,364.72	123.15 (11.08×)	87.3	170.1	10.24 (133.27×)	5.8 (↓93.36%)	143.0 (↓15.93%)
	30	1,405.17	199.94 (7.03×)	162.4	348.3	16.99 (82.71×)	8.4 (↓94.83%)	201.1 (↓42.26%)
Vim	1	408.48	28.63 (14.27×)	22.0	66.0	1.22 (334.82×)	1.4 (↓93.64%)	30.8 (↓53.33%)
	10	405.05	198.07 (2.04×)	242.9	259.3	17.01 (23.81×)	16.0 (↓93.41%)	228.9 (↓11.72%)
	20	418.33	373.37 (1.12×)	392.4	436.9	36.51 (11.46×)	28.2 (↓92.81%)	336.2 (↓23.05%)
	30	413.61	464.57 (0.89×)	525.0	854.6	61.35 (6.74×)	48.4 (↓90.78%)	730.6 (↓14.51%)
OpenSSL	1	526.37	7.73 (68.09×)	6.4	15.1	1.45 (363.01×)	1.2 (↓81.25%)	13.7 (↓9.27%)
	10	530.40	87.88 (6.04×)	87.6	147.5	17.7 (29.97×)	16.7 (↓80.94%)	87.4 (↓40.75%)
	20	523.79	320.02 (1.64×)	180.9	233.6	42.72 (12.26×)	31.2 (↓82.75%)	121.4 (↓48.03%)
	30	531.42	680.41 (0.78×)	291.4	342.5	59.05 (9.00×)	47.1 (↓83.84%)	154.3 (↓54.95%)
Postgre-SQL	1	4,578.48	30.31 (151.06×)	7.5	29.7	4.71 (972.08×)	1.4 (↓81.33%)	19.1 (↓35.69%)
	10	4,571.15	192.68 (23.72×)	55.7	148.3	22.53 (202.89×)	6.8 (↓87.79%)	133.0 (↓10.32%)
	20	4,606.47	368.44 (12.50×)	94.1	201.2	35.12 (131.16×)	8.9 (↓90.54%)	187.5 (↓6.81%)
	30	4,588.14	564.7 (8.12×)	136.8	347.0	53.2 (86.24×)	14.7 (↓89.25%)	261.8 (↓24.55%)
Avg.	1	1,407.59	26.48 (53.16×)	19.3	43.8	2.16 (651.66×)	1.36 (↓92.75%)	23.46 (↓89.25%)
	10	1,411.29	129.78 (10.87×)	129.5	243.0	13.08 (107.90×)	9.66 (↓92.51%)	192.92 (↓20.62%)
	20	1,408.67	271.16 (5.19×)	337.4	451.7	25.37 (55.53×)	16.44 (↓95.14%)	353.72 (↓21.70%)
	30	1,414.14	429.42 (3.29×)	464.9	756.5	38.77 (36.48×)	26.18 (↓94.36%)	519.62 (↓31.32%)

The "**i**" column presents the commit interval. The "**#SCC**" presents the average number of SCC detections performed per run. The "**#Node**" presents the average number of pointer nodes processed per run. The red font indicates the increased speed compared to the full analysis. The blue font indicates the percentage reduction achieved by our approach compared to IPA for the corresponding metric.

approach [16] with our incremental function summary analysis algorithm, as many existing methods [2, 49, 56, 57, 66, 80] for incremental analysis of function summaries are largely based on this classic idea.

We select extensive commit versions from the target projects' GitHub repositories to evaluate these two modules. We first choose different commit intervals ($i = 1, 10, 20$, and 30 commits), and for each interval, we randomly select 20 versions for 10 sets of evaluations. For instance, in the $i = 20$ interval scenario, one feasible experiment would be: initializing with the full analysis of version c1, computing code changes between versions c1 and c21 (which are separated by 20 effective commits) as input for incremental analysis, and running the incremental analysis to obtain the analysis results for version c21. We record the average data for each algorithm across the 10 sets of experiments for each interval.

Results. Tables 5 and 6 display the specific data for the comparative evaluations of the two modules.

Table 5 lists the experimental data of our incremental PTA algorithm, the full algorithm, and the existing incremental algorithm (IPA) across different commit intervals. Column 2 shows the four commit intervals (denoted as "**i**") we selected. Column 3 displays the time for full PTA. Columns 4–6 provide the data of IPA, while columns 7–9 are for ours. "**Time (s)**" indicates the average runtime, "**#SCC**" represents the average number of SCC detections performed per run, and "**#Node**" shows the average number of pointer nodes processed per run. Overall, our incremental algorithm

Table 6. Statistics of Different Mod-Ref Analyses under Each Commit Interval

Project	i	Full	Reset-Recompute			Ours		
		Time (s)	Time (s)	#Func	#Elem	Time (s)	#Func	#Elem
Tmux	1	27.89	21.73 (1.28×)	>600	707K	6.28 (4.44×)	<50 (↓>91.67%)	<1K (↓>99.86%)
	10	28.36	25.05 (1.13×)	>700	721K	15.73 (1.80×)	<50 (↓>92.86%)	36K (↓95.01%)
	20	26.81	26.62 (1.01×)	>750	750K	17.44 (1.54×)	<100 (↓>86.67%)	50K (↓93.33%)
	30	29.70	28.27 (1.05×)	>750	764K	19.08 (1.56×)	<200 (↓>73.33%)	65K (↓91.49%)
Git	1	10,586.48	2,346.71 (4.51×)	>3K	7M	922.93 (11.47×)	<300 (↓>90.00%)	30K (↓99.57%)
	10	10,834.30	7,973.45 (1.36×)	>8K	90M	1,069.54 (10.13×)	<1K (↓>87.50%)	1M (↓98.89%)
	20	10,231.48	8,921.93 (1.15×)	>9K	372M	1,677.71 (6.10×)	<2K (↓>77.78%)	6M (↓98.39%)
	30	10,508.98	9,279.50 (1.13×)	>9K	414M	1,830.04 (5.74×)	<2K (↓>77.78%)	7M (↓98.31%)
Vim	1	1,099.69	948.36 (1.16×)	>4K	27M	184.82 (5.95×)	<100 (↓>97.50%)	10K (↓99.96%)
	10	1,143.59	1,072.24 (1.07×)	>5K	31M	205.05 (5.58×)	<200 (↓>96.00%)	30K (↓99.90%)
	20	1,250.62	1,100.56 (1.14×)	>5K	34M	216.09 (5.79×)	<200 (↓>96.00%)	46K (↓99.86%)
	30	1,227.19	1,163.76 (1.05×)	>5K	36M	260.28 (5.71×)	<200 (↓>96.00%)	52K (↓99.86%)
OpenSSL	1	3,350.68	1,988.75 (1.68×)	>7K	40M	427.92 (7.83×)	<100 (↓>98.57%)	15K (↓99.96%)
	10	3,405.40	2,223.62 (1.53×)	>8K	56M	439.31 (7.75×)	<500 (↓>93.75%)	20K (↓99.96%)
	20	3,283.94	2,912.77 (1.13×)	>8K	57M	601.40 (5.46×)	<1K (↓>87.50%)	41K (↓99.93%)
	30	3,611.83	3,084.82 (1.17×)	>8K	58M	626.95 (5.76×)	<1K (↓>87.50%)	49K (↓99.92%)
Postgre-SQL	1	12,236.58	8,192.74 (1.49×)	>14K	74M	1,077.23 (11.36×)	<100 (↓>96.67%)	6K (↓99.99%)
	10	12,652.01	9,427.14 (1.34×)	>15K	78M	1,186.17 (10.67×)	<500 (↓>96.67%)	31K (↓99.96%)
	20	12,874.50	11,314.30 (1.14×)	>16K	80M	1,345.09 (9.57×)	<1K (↓>93.75%)	40K (↓99.98%)
	30	12,500.23	12,410.2 (1.01x)	>16K	80M	1,405.70 (8.89×)	<1K (↓>93.75%)	40K (↓99.98%)
Avg.	1	5,460.26	2,699.66 (2.02×)	>5.9K	30M	523.84 (10.42×)	<210 (↓>94.88%)	12K (↓99.87%)
	10	5,612.73	4,144.30 (1.35×)	>7.3K	51M	583.16 (9.62×)	<450 (↓>93.35%)	223K (↓98.74%)
	20	5,533.47	4,855.24 (1.14×)	>7.8K	125M	771.55 (7.17×)	<860 (↓>88.34%)	1.2M (↓98.30%)
	30	5,575.59	5,193.31 (1.07×)	>7.8K	134M	828.41 (6.73×)	<880 (↓>85.67%)	1.4M (↓97.91%)

The "**i**" column presents the commit interval. The "**#Func**" presents the average number of functions whose Mod/Ref summaries changed per run. The "**#Node**" presents the average number of Mod/Ref summary elements added or deleted per run. The red font indicates the increased speed compared to the full analysis. The blue font indicates the percentage reduction achieved by our approach compared to reset-recompute for the corresponding metric.

demonstrates better performance than both full PTA and IPA across all commit intervals. On average, our incremental PTA is over 70× faster than the full analysis and more than 10× faster than IPA. The recorded SCC detection counts and node processing counts effectively explain why our incremental PTA algorithm is more efficient: Our method significantly reduces the frequency of these two key time-consuming operations. Notably, our approach reduces SCC detection counts by over 90% compared to IPA, allowing it to scale well even in scenarios with larger commit intervals and more changes. We observe that IPA's performance deteriorates with increasing commit intervals, as more code changes lead to more SCC detections and pointer node processing. In the evaluation of Tmux, for intervals of 20 and 30, IPA is already slower than full analysis due to the large volume of changes making IPA's SCC detection and node processing counts approach or even exceed those of full analysis, while our method significantly reduces the number of such operations, maintaining over 40× faster performance than full analysis.

Table 6 lists the experimental data of our incremental MRA, the full analysis, and the traditional reset-recompute method across different commit intervals. Similarly, column 2 shows the four commit intervals (denoted as "**i**"), and column 3 shows the time for full MRA. Columns 4–6 contain data for the reset-recompute method, while columns 7–9 are for our method. "**Time (s)**" indicates the average runtime, "**#Func**" denotes the number of functions whose Mod/Ref summaries changed during incremental analysis (i.e., for reset-recompute, it indicates the number of functions whose summaries are reset and recomputed, and for our method, it indicates the number of

functions whose summary elements are deleted or added), "**#Elem**" represents the number of Mod/Ref summary elements added or deleted during incremental analysis. Overall, our incremental algorithm also significantly outperforms both full analysis and the reset-recompute method. On average, our incremental MRA is over 8× faster than full analysis and more than 6× faster than reset-recompute. The data under "**#Func**" and "**#Elem**" also effectively explain why our incremental MRA is more efficient: We process fewer functions and change fewer summary elements. Note that our incremental MRA's analysis results are consistent with those of full analysis and reset-recompute, meaning we significantly reduce redundant change propagation and processing. The classic reset-recompute method simply resets all reachable function summaries and then recomputes them, a process whose function and summary counts can approach those of full analysis in scenarios with larger commit intervals. However, our experimental results show that not as many functions are affected by changes in real-world code commit scenarios as reset-recompute suggests. Our method uses intra-procedural summaries to filter out redundant inter-procedural summary propagation, thus performing more efficiently. Nonetheless, even with such a substantial reduction in the number of functions and summary elements processed, our incremental MRA still incurs an average cost of nearly 600 seconds, mainly because our method still needs to traverse all functions at least once again during the summary element re-addition phase to determine if summaries have changed. Despite this, compared to full analysis and reset-recompute, the performance of our incremental MRA is still quite remarkable.

5.3 The Performance of SILVA in Analyzing Varying Scales of Code Changes

Experimental Setup. We evaluate the performance of SILVA across scenarios with varying scales of code changes and try to identify the "capability boundary" of our incremental approach—that is, in what scenarios our incremental approach has an advantage over the full SVFA. We select the smallest, medium, and largest projects from the benchmark (namely Tmux, Vim, and PostgreSQL), with their full SVFA times being on the order of hundreds, thousands, and tens of thousands of seconds, respectively. Then we obtain tags from the GitHub repositories of these projects, compute the lines of code changed between different tags, and ensure that these changes cover a wide range of sizes as much as possible. To obtain changes spanning tens or even hundreds of thousands of lines, we perform git diff across multiple tags. For each project, we conduct at least 10 sets of experiments, i.e., incremental analysis on at least 10 different scales of changes. Moreover, we run SILVA three times on each change scale and recorded the average time.

Results. Figure 15 presents the time taken by SILVA for incremental SVFA on Tmux, Vim, and PostgreSQL across various scenarios of code line changes. The general trend within each program is that the more changed LOC, the longer the analysis time. For the three target projects, a common pattern observed is that the analysis time initially increases rapidly with the number of lines of code changed. We investigate the underlying reasons: When the code changes are minimal, only a few constraint edges are processed in the incremental PTA module, meaning that only a few SCCs need maintenance. However, as the scale of changes begins to grow, both the scale and number of SCCs being processed surge dramatically. Especially, as the number of SCCs disrupted due to deleted statements increases, incremental PTA thus spends a significant amount of time on the recovery of SCCs and the propagation of changed points-to sets. This leads to extensive changes in the intra-Mod/Ref sets of many functions, resulting in a rapid increase in the time required for MRA.

Furthermore, these curves exhibit fluctuations because the analysis time is not solely related to the number of lines of code changed. Many factors affect the time required for incremental analysis [65, 81, 82], such as the number of changed pointer statements, the number of changed

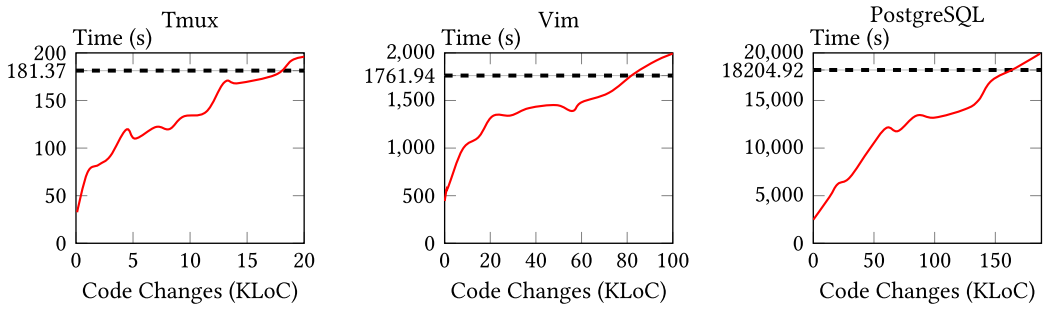


Fig. 15. Code Change Analysis Efficiency Curves for SILVA when analyzing Tmux, Vim, and PostgreSQL. The red curve indicates the incremental SVFA time as it varies with the number of lines of code changed, while the black dashed line marks the time taken for SVF's full SVFA. Here, the code changes refer to the sum of added and deleted lines of C/C++ code between two versions, as determined by git diff, excluding comments.

memory objects, the number of changed functions, and the complexity of the changes in the code. This presents an interesting and challenging research topic that precisely determines when to use incremental analysis versus full analysis and inspire future researchers in incremental analysis to improve efficiency by focusing on the factors that bottleneck incremental analysis.

However, when considering just the number of lines of code changed, the trend in analysis time is already sufficiently clear. For these three projects of varying sizes, incremental SVFA times converge with full analysis around code changes of approximately 17K, 83K, and 160K lines, respectively. In terms of the percentage of changes for each project, a 15–25% code change is the threshold between choosing incremental over full analysis. We advocate using incremental analysis in scenarios of frequent code commits or continuous integration and delivery pipelines, where code changes of thousands, hundreds, or even tens of lines are more common [82], rather than changes in the tens of thousands. In such scenarios, SILVA's incremental analysis offers a significant advantage over full analysis, with speed improvements of more than 1×.

5.4 The Correctness of Our Implementations

Although we provide theoretical proof of the correctness of our algorithms in Section 4, we still take measures to verify the correctness of our implementation. Specifically, for all versions of the projects used in the aforementioned evaluations, we also run a full SVFA using SVF and record its results, and we compare the points-to sets of all pointers, the call graph of the program, and the Mod-Ref sets of each function in the analysis results with those of SILVA. Our experiments confirm that the incremental analysis results of SILVA are consistent with the full analysis results of SVF in all cases.

Additionally, we compare the analysis results of our reproduced methods (i.e., IPA and reset-recompute algorithms) with those of the full analysis to ensure the correctness of the reproductions. Furthermore, we compile some Java programs used in the experimental section of the IPA algorithm paper [33] into LLVM IR and evaluate them using our reproduced IPA. The evaluation results indicate that the acceleration effect of our reproduced IPA, compared to the full analysis, is almost identical to that reported in the original IPA paper, further ensuring the correctness of our re-implementation.

5.5 Discussion

5.5.1 Initialization of Incremental Analysis. It should be clarified that the analysis results used to initialize SILVA are not obtained from the original full analysis of SVF. We made slight modifications

Table 7. The Time Required for the SVFA That Is Used to Initialize SILVA

Project	Tmux	Vim	Git	OpenSSL	PostgreSQL
$SVFA_{init}$(s)	199.54 (↑10.01%)	1,800.32 (↑2.18%)	13,817.36 (↑9.24%)	4,420.86 (↑5.66%)	19,506.22 (↑7.15%)

"↑" shows the percentage increase in time spent on the full analysis used for initialization compared to the original full analysis.

to the analysis process of the original SVF to compute and retain certain information necessary for incremental analysis. For instance, in incremental PTA, we preserve the internal structures of all SCCs on the constraint graph to accurately restore those SCCs that might be disrupted by edge deletions during the deletion process. This information is discarded in the original SVF's full analysis to reduce memory consumption, as the scenario of edge deletion does not occur in a full analysis process.

Table 7 shows the time required for the SVFA used to initialize SILVA for each project. We observe that the time expenditure for the full SVFA used for initialization is at most 1.1× that of the original SILVA's full analysis. The additional overhead is acceptable because it is one-off costs only: Assuming the incremental analysis is deployed on a continuous-integration server and executed with each code change, the information preserved during the initial setup will be available for use in all subsequent incremental analyses.

5.5.2 Memory Usage. While SILVA offers significant advantages in analysis efficiency, it necessitates some tradeoffs in other areas. For full analysis, memory usage can often be reduced during the analysis process by discarding intermediate results that are no longer needed in subsequent analyses. However, incremental analysis must maintain all intermediate results related to the final outcome throughout the entire analysis process. Moreover, in certain analysis modules, it is necessary to store intermediate data structures (e.g., call graphs) for both the new and old versions of the program. Currently, SILVA faces issues with relatively high memory usage. The memory consumption of SILVA is on average nearly 25.2% higher than that of full analysis tools SVF, while the worst-case scenario we observed for SILVA is 1.5 times that of SVF. Memory issues have become a major obstacle for SILVA to analyze larger-scale programs further, which is a problem also faced by other incremental analysis efforts [32, 33, 65]. To reduce memory usage, one approach is to delete some intermediate results that are temporarily not needed during analysis. However, if these intermediate results are needed for subsequent analysis, they must be recalculated, incurring a cost in terms of time. Currently, SILVA does not support such a feature. Exploring the tradeoff between discarding intermediate computation results to reduce memory usage and better balancing memory and time could be an intriguing research direction for future static analysis.

5.5.3 Threats to Validity. There are three main threats to the validity of our evaluation: (1) the limited number of C/C++ projects used for evaluation, (2) the limited number of commits selected for evaluation, which may not accurately reflect the effectiveness of SILVA across all types of code changes, and (3) the correctness of the implementation.

To mitigate the first threat, we selected a set of real-world projects from GitHub with varying code sizes and frequent code commits. These five projects are widely used in recent C/C++ program analysis research evaluations [4, 34, 52], which partially reflects their validity.

For the second threat, we designed two experimental scenarios: consecutive commits (Section 5.1) and non-consecutive commits (Section 5.3), to cover a wide range of code changes. Additionally, we manually reviewed all the commits used for evaluation to ensure that they include various types of statement deletions and insertions, which helps alleviate this threat to some extent.

Finally, we selected the widely used open source VFA framework, SVF [60], as the implementation framework for SILVA, and use the full analysis in SVF as a baseline for comparison. We manually verified the analysis results to ensure consistency between the full analysis and the various incremental methods. Through these measures, we minimized the threat to the correctness of the implementation.

6 Limitations and Future Work

In this section, we discuss the limitations of SILVA and some potential future work.

First, SILVA currently only supports "layered" program analysis and defect detection techniques where PTA and MRA are two prerequisite modules. Thus, SILVA is not directly applicable to some existing "holistic" SVFAs, such as Pinpoint [52]. Designing incremental algorithms for holistic SVFA is a possible future direction. Such analyses use a demand-driven approach to construct a **symbolic expression graph (SEG)**, computing pointer aliasing relations only during the value-flow path analysis. We can leverage the idea of incremental analysis to update the SEG and value-flow path information incrementally when code changes occur, rather than recomputing them from scratch.

Second, both incremental algorithms in SILVA follow the traditional "delete first, add later" strategy used in incremental analysis research. However, as mentioned earlier, for certain changes, carefully arranging the order of deletion and insertion can further reduce redundant computations. For example, in PTA, if a CopyEdge can be derived from both a deleted StoreEdge and a newly inserted LoadEdge, there is no need to redundantly delete and then re-add the CopyEdge. By fully utilizing binding sets and exploring potential compensatory changes, we can make the incremental analysis more efficient. We believe this is an interesting avenue for future work.

Finally, the incremental scenario discussed in this article is based on large-scale software development environments where code changes are made through repository commits. Another common incremental scenario is when developers update code in real-time within an **integrated development environment (IDE)**, with the analysis providing immediate feedback on newly introduced issues. Currently, SILVA has not been integrated into IDEs, and its performance is not yet sufficient to provide developers with an imperceptible analysis time. A meaningful direction for future work is to investigate more efficient incremental analysis techniques, enabling even resource-intensive analyses like VFA to be integrated into IDEs, thereby helping software developers to more efficiently produce more reliable and secure code.

7 Related Work

In this section, we focus on discussing two lines of related work, touching upon our topic—VFA and incremental analysis.

7.1 VFA

Many existing VFA [8, 10, 36, 44, 53, 59, 60] rely on a pre-computed PTA to construct the value-flow, a design often referred to as "layered." For example, FASTCHECK [7] uses Steensgaard's PTA [54] to construct the value-flow graph of a program, while SABER [62, 63] leverages Andersen's analysis. SABER tracks the flow of values from allocation to free sites using a sparse graph representation that captures def-use chains and value-flows via assignments for both top-level and address-taken pointers, which is more accurate than FASTCHECK. In contrast to the "layered" design, Pinpoint [52] provides a "holistic" design of SVFA that can build precise data dependence efficiently and keep fully path-sensitive for bug finding. Cai et al. [4] present a VFA framework for concurrent programs to find inter-thread value-flow bugs. The algorithm used in CANARY iteratively identifies interference dependence by only reasoning the shared memory locations and avoids using a full and expensive PTA as a prior, while remaining path-sensitive. ANCHOR [69] further improves the

accuracy of VFA. Compared to other value-flow analyses that do not support precise analysis of containers and smash containers into the sets of objects, it is the first practical VFA supporting precise container analysis. Yao et al. [76] introduce a scalable path- and context-sensitive data dependence analysis that mitigates aliasing-path-explosion by decomposing the problem into a semi-path-sensitive value-flow graph construction and a demand-driven phase for resolving transitive data dependence with path-sensitive pointer information. Some studies aim to improve the efficiency of propagating dataflow facts or value-flows based on IFDS/IDE frameworks. For example, MergeDroid [15] employs a simple yet effective merge-and-replay algorithm to consolidate equivalent value-flows. SparseDroid [18] leverages the sparsity principles of SVFA to sparsify IFDS-based taint analysis, thereby enhancing performance. In addition, Karakaya and Bodden [22] present a more generic Sparse IDE algorithm, which efficiently solves not only IFDS-based reachability problems but also IDE problems that require value computation.

Our work is orthogonal to these efforts that improve VFA. Standing from the perspective of large-scale evolving software, we start with code changes and leverage incremental methods, enabling VFA to scale to large-scale software. In principle, our incremental approach and principles can be extended to other VFA techniques.

7.2 Incremental Analysis

Recently, incremental analysis has gained significant attention due to its ability to quickly provide the latest analysis results in response to code changes. It has widespread applications, such as termination analysis [19], bug finding [11], clone detection [42] and verification [27, 77]. Here, we categorize the related work on incremental analysis into two discussions: incremental PTA and other incremental analysis.

7.2.1 Incremental PTA. Most existing incremental pointer analyses [23, 48, 49, 78, 80] are derived from the DRed algorithm [16]. Nevertheless, they often fall short in accurately addressing dynamic changes in method calls, or they struggle to scale effectively to large-scale codebases due to redundant computations. IPA [31, 33] improves DRed by observing the change-locality property in the Pointer Assignment Graph for Java programs. Inspired by IPA, we develop an incremental PTA that supports C/C++ language features (e.g., pointer dereference and function pointer). Our algorithm adopts a merge-then-process approach, which achieves further acceleration compared to IPA. We also resolve the problem of removing multi-source derived copy constraint edge, which is ignored in IPA. SHARP [32] extends IPA to consider contexts and reduces the redundancy in change propagation when deleting method calls. Theoretically, our incremental PTA algorithm could also adopt SHARP's approach to achieve context-sensitivity. Some incremental PTA is based on CFL-reachability. Shang et al. [50] present EMU, which enables developers to perform pointer-related queries in programs undergoing constant changes in IDEs. However, EMU assumes a prebuilt call graph of the program, which does not hold for scenarios in which the call graph itself can be modified by code changes. Furthermore, it does not preserve precision but rather computes a less precise result than the full analysis. Lu et al. [37] present an incremental approach for demand-driven context-sensitive points-to analyses based on CFL-reachability. By tracing the CFL-reachable paths traversed in computing points-to sets, their approach can precisely identify and recompute on demand only the points-to sets affected by the program changes made. Krishna et al. [26] develop a novel dynamic algorithm for bidirected Dyck-CFL reachability, which incrementally maintains Dyck SCCs in bidirectional graphs to update reachabilities when code changes, avoiding the need to start from scratch. Our incremental algorithm is based on Andersen's analysis, which is another widely used approach focused on the whole program and is more suitable as a pre-analysis for inter-procedural value-flow construction. Some efficient incremental PTA algorithms [25, 72] focus

only on code additions and cannot handle code deletions. This makes them difficult to apply in real code change scenarios. In contrast, our approach handles both additions and deletions and has been evaluated for effectiveness in real code commit scenarios.

7.2.2 Other Incremental Analysis. In addition to PTA, other static analysis techniques have also made progress in incremental analysis [38, 40, 41, 68]. IncA [66] proposes a domain-specific language to incrementally update program analysis results based on graph pattern matching. REVISER [2] proposes an inter-procedural dataflow analysis based on the IFDS/IDE framework [47] where a dataflow problem is transformed into a graph reachability problem over exploded supergraph. Reviser adopts a reset-recompute approach: It first collects the changed nodes and their affected nodes, then applies a *clear-and-propagate* strategy to reset and recompute the corresponding IDE results for these nodes. Some incremental dataflow analysis algorithms [20, 21, 28, 79] are based on function summaries, which can incrementally update function summaries and analysis results on the function call graph based on code changes. For example, the method by Jana et al. [20] prioritizes collecting the set of functions that have been edited, and then directly recomputes the summaries of these functions. If the newly computed summaries differ from the old ones, the summaries of the caller functions are also recomputed, and this process continues recursively. This approach is slightly more efficient than traditional reset-recompute methods, as it avoids resetting the summaries of all functions that are reachable through changes. However, it still cannot avoid recomputing the summaries of some functions. In contrast, our incremental function summary analysis takes a finer-grained approach to incremental updates, where it propagates changes to update the summary, rather than recomputing the entire summary. Moreover, we introduce an early-stop strategy for function-summary propagation, which divides function summaries into intra-procedural and inter-procedural summaries, and uses intra-procedural summaries to filter out propagated inter-procedural summary changes. This strategy allows the propagation of summary changes to stop as early as possible. In addition, the method by Jana et al. [20] assumes that the call chain never changes. When the call chain indeed changes, they resort to full analysis rather than incremental analysis. In contrast, SILVA's incremental analysis is capable of correctly handling such cases. Some incremental approaches focus on the construction of the call graph. For example, Zhao et al. [82] present an incremental call graph construction algorithm based on the CHA [9] call graph construction. The incremental approach we used in the PTA can also update the call graph on-the-fly while updating the points-to information, and the resulting call graph is more accurate.

Incremental computation has been extensively discussed in the domain of Datalog evaluation. Intensive research [16, 41, 64, 65] focuses on incremental computation using Datalog. These Datalog-based incremental methods are applicable to a variety of static analyses but do not fully leverage the unique neighbor node properties inherent in PTA or utilize the characteristics of inter/intra-summaries in MRA to further reduce redundant computations. There is also existing work [33] that indicates Datalog engines are still inefficient in handling the incremental deletion of the PTA facts.

Some studies have proposed abstract interpretation frameworks that support incremental processing [56, 57]. These methods are based on the demanded abstract interpretation graph or demanded summarization graph, fundamentally performing incremental reset and recomputation of summaries on these graph structures. However, it is unclear whether these methods can scale to large-scale programs (hundreds of thousands or millions of lines of code). In contrast, our incremental approach is dedicated to scaling complex value-flow analyses to large-scale programs.

8 Conclusion

We have conducted an empirical study to investigate the bottleneck of layered SVFA and proposed SILVA, the first incremental layered SVFA that scales to large, real-world programs. SILVA employs

our two novel incremental algorithms for the two bottleneck modules: the PTA and the MRA. Our evaluation of SILVA on real-world C/C++ large projects shows that SILVA achieves a nearly 7× speedup compared to SVF, a state-of-the-art layered SVFA. Additionally, our novel incremental PTA and incremental MRA are 12× and 5× faster, respectively, than existing methods. Our findings also indicate that even on code changes amounting to 10K lines, SILVA still outperforms SVF, demonstrating performance that surpasses typical scenarios.

Acknowledgment

The authors would like to thank the anonymous reviewers for their comments and suggestions on the article.

References

[1] Lars Ole Andersen. 1994. *Program Analysis and Specialization for the C Programming Language*. Ph.D. Dissertation. Citeseer.

[2] Steven Arzt and Eric Bodden. 2014. Reviser: Efficiently updating IDE-/IFDS-based data-flow analyses in response to incremental program changes. In *Proceedings of the 36th International Conference on Software Engineering*, 288–298.

[3] Domagoj Babic and Alan J. Hu. 2008. Calysto: Scalable and precise extended static checking. In *Proceedings of the 30th International Conference on Software Engineering*, 211–220.

[4] Yuandao Cai, Peisen Yao, and Charles Zhang. 2021. Canary: Practical static detection of inter-thread value-flow bugs. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 1126–1140.

[5] Xiao Cheng, Haoyu Wang, Jiayi Hua, Guoai Xu, and Yulei Sui. 2021. DeepWukong: Statically detecting software vulnerabilities using deep graph neural network. *ACM Transactions on Software Engineering and Methodology* 30, 3 (2021), 1–33.

[6] Xiao Cheng, Jiawei Wang, and Yulei Sui. 2024. Precise sparse abstract execution via cross-domain interaction. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–12.

[7] Sigmund Cherem, Lonnie Princehouse, and Radu Rugina. 2007. Practical memory leak detection using guarded value-flow analysis. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 480–491.

[8] Manuvir Das, Sorin Lerner, and Mark Seigle. 2002. ESP: Path-sensitive program verification in polynomial time. In *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation*, 57–68.

[9] Jeffrey Dean, David Grove, and Craig Chambers. 1995. Optimization of object-oriented programs using static class hierarchy analysis. In *Proceedings of the 9th European Conference on Object-Oriented Programming (ECOOP '95)*. Springer, 77–101.

[10] Nurit Dor, Stephen Adams, Manuvir Das, and Zhe Yang. 2004. Software validation via scalable path-sensitive value flow analysis. *ACM SIGSOFT Software Engineering Notes* 29, 4 (2004), 12–22.

[11] Alexandru Dura, Christoph Reichenbach, and Emma Söderberg. 2021. JavaDL: Automatically incrementalizing Java bug pattern detection. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (2021), 1–31.

[12] Manuel Fähndrich, Jeffrey S. Foster, Zhendong Su, and Alexander Aiken. 1998. Partial online cycle elimination in inclusion constraint graphs. In *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation*, 85–96.

[13] Josselin Feist, Laurent Mounier, and Marie-Laure Potet. 2014. Statically detecting use after free on binary code. *Journal of Computer Virology and Hacking Techniques* 10, 3 (2014), 211–217.

[14] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. 1987. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems* 9, 3 (1987), 319–349.

[15] Yujiang Gui, Dongjie He, and Jingling Xue. 2023. Merge-Replay: Efficient IFDS-based taint analysis by consolidating equivalent value flows. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 319–331.

[16] Ashish Gupta, Inderpal Singh Mumick, and Venkatramanan Siva Subrahmanian. 1993. Maintaining views incrementally. *ACM SIGMOD Record* 22, 2 (1993), 157–166.

[17] Ben Hardekopf and Calvin Lin. 2007. The ant and the grasshopper: Fast and accurate pointer analysis for millions of lines of code. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 290–299.

[18] Dongjie He, Haofeng Li, Lei Wang, Haining Meng, Hengjie Zheng, Jie Liu, Shuangwei Hu, Lian Li, and Jingling Xue. 2019. Performance-boosting sparsification of the IFDS algorithm with applications to taint analysis. In *Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 267–279.
[19] Fei He and Jitao Han. 2020. Termination analysis for evolving programs: An incremental approach by reusing certified modules. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–27.
[20] Anushri Jana, Bharti Chimdyalwar, Shrawan Kumar, and R. Venkatesh. 2022. Fast analysis of evolving software systems. In *Proceedings of the 2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 49–54.
[21] Anushri Jana, Ankita Khadsare, Bharti Chimdyalwar, Shrawan Kumar, Vaidehi Ghime, and R. Venkatesh. 2021. Fast change-based alarm reporting for evolving software systems. In *Proceedings of the 2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 546–556.
[22] Kadiray Karakaya and Eric Bodden. 2024. Symbol-specific sparsification of interprocedural distributive environment problems. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–12.
[23] George Kastrinis and Yannis Smaragdakis. 2013. Efficient and effective handling of exceptions in Java points-to analysis. In *Proceedings of the International Conference on Compiler Construction*. Springer, 41–60.
[24] Uday Khedker, Amitabha Sanyal, and Bageshri Sathe. 2017. *Data Flow Analysis: Theory and Practice*. CRC Press.
[25] Jakob Krainz and Michael Philippsen. 2017. Diff graphs for a fast incremental pointer analysis. In *Proceedings of the 12th Workshop on Implementation, Compilation, Optimization of Object-Oriented Languages, Programs and Systems*, 1–10.
[26] Shankaranarayanan Krishna, Aniket Lal, Andreas Pavlogiannis, and Omkar Tuppe. 2024. On-the-fly static analysis via dynamic bidirected Dyck reachability. *Proceedings of the ACM on Programming Languages* 8, POPL (2024), 1239–1268.
[27] Shriram Krishnamurthi and Kathi Fisler. 2007. Foundations of incremental aspect model-checking. *ACM Transactions on Software Engineering and Methodology* 16, 2 (2007), 7–es.
[28] Padmanabhan Krishnan, Rebecca O'Donoghue, Nicholas Allen, and Yi Lu. 2019. Commit-time incremental analysis. In *Proceedings of the 8th ACM SIGPLAN International Workshop on State of the Art in Program Analysis*, 26–31.
[29] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO '04)*. IEEE, 75–86.
[30] Yuanbo Li, Qirun Zhang, and Thomas Reps. 2022. Fast graph simplification for interleaved-Dyck reachability. *ACM Transactions on Programming Languages and Systems* 44, 2 (2022), 1–28.
[31] Bozhen Liu and Jeff Huang. 2018. D4: Fast concurrency debugging with parallel differential analysis. *ACM SIGPLAN Notices* 53, 4 (2018), 359–373.
[32] Bozhen Liu and Jeff Huang. 2022. SHARP: Fast incremental context-sensitive pointer analysis for Java. *Proceedings of the ACM on Programming Languages* 6, OOPSLA1 (2022), 1–28.
[33] Bozhen Liu, Jeff Huang, and Lawrence Rauchwerger. 2019. Rethinking incremental and parallel pointer analysis. *ACM Transactions on Programming Languages and Systems* 41, 1 (2019), 1–31.
[34] Peiming Liu, Yanze Li, Brad Swain, and Jeff Huang. 2022. PUS: A fast and highly efficient solver for inclusion-based pointer analysis. In *Proceedings of the 44th International Conference on Software Engineering*, 1781–1792.
[35] Pengcheng Liu, Yifei Lu, Wenhua Yang, and Minxue Pan. 2023. VALAR: Streamlining alarm ranking in static analysis with value-flow assisted active learning. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1940–1951.
[36] V. Benjamin Livshits and Monica S. Lam. 2003. Tracking pointers with path and context sensitivity for bug detection in C programs. In *Proceedings of the 9th European Software Engineering Conference Held Jointly with 11th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 317–326.
[37] Yi Lu, Lei Shang, Xinwei Xie, and Jingling Xue. 2013. An incremental points-to analysis with CFL-reachability. In *Proceedings of the 22nd International Conference on Compiler Construction (CC '13), Held as Part of the European Joint Conferences on Theory and Practice of Software (ETAPS '13)*. Springer, 61–81.
[38] Abu Naser Masud. 2022. Fast and incremental computation of weak control closure. In *Proceedings of the International Static Analysis Symposium*. Springer, 325–349.
[39] Connor McLaughlin and Yi Lu. 2024. Multi-class vulnerability prediction using value flow and graph neural networks. *Neural Computing and Applications* 36 (2024), 15869–15891.
[40] Scott McPeak, Charles-Henri Gros, and Murali Krishna Ramanathan. 2013. Scalable and incremental software bug detection. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*, 554–564.
[41] Boris Motik, Yavor Nenov, Robert Piro, and Ian Horrocks. 2015. Incremental update of datalog materialisation: The backward/forward algorithm. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 29.
[42] Tung Thanh Nguyen, Hoan Anh Nguyen, Jafar M. Al-Kofahi, Nam H. Pham, and Tien N. Nguyen. 2009. Scalable and incremental clone detection for evolving software. In *Proceedings of the 2009 IEEE International Conference on Software Maintenance*. IEEE, 491–494.

[43] Esko Nuutila and Eljas Soisalon-Soininen. 1994. On finding the strongly connected components in a directed graph. *Information Processing Letters* 49, 1 (1994), 9–14. DOI: https://doi.org/10.1016/0020-0190(94)90047-7
[44] Hakjoo Oh, Kihong Heo, Wonchan Lee, Woosuk Lee, and Kwangkeun Yi. 2012. Design and implementation of sparse global analyses for C-like languages. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*, 229–238.
[45] David J. Pearce, Paul H. J. Kelly, and Chris Hankin. 2007. Efficient field-sensitive pointer analysis of C. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 30, 1 (2007), 4–es.
[46] Fernando Magno Quintao Pereira and Daniel Berlin. 2009. Wave propagation and deep propagation for pointer analysis. In *Proceedings of the 2009 International Symposium on Code Generation and Optimization*. IEEE, 126–135.
[47] Thomas Reps, Susan Horwitz, and Mooly Sagiv. 1995. Precise interprocedural dataflow analysis via graph reachability. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 49–61.
[48] Diptikalyan Saha and C. R. Ramakrishnan. 2005. Incremental and demand-driven points-to analysis using logic programming. In *Proceedings of the 7th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming*, 117–128.
[49] Diptikalyan Saha and C. R. Ramakrishnan. 2005. Symbolic support graph: A space efficient data structure for incremental tabled evaluation. In *Proceedings of the International Conference on Logic Programming*. Springer, 235–249.
[50] Lei Shang, Yi Lu, and Jingling Xue. 2012. Fast and precise points-to analysis with incremental CFL-reachability summarisation: Preliminary experience. In *Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering*, 270–273.
[51] Qingkai Shi, Rongxin Wu, Gang Fan, and Charles Zhang. 2020. Conquering the extensional scalability problem for value-flow analysis frameworks. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 812–823.
[52] Qingkai Shi, Xiao Xiao, Rongxin Wu, Jinguo Zhou, Gang Fan, and Charles Zhang. 2018. Pinpoint: Fast and precise sparse value flow analysis for million lines of code. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 693–706.
[53] Gregor Snelting, Torsten Robschink, and Jens Krinke. 2006. Efficient path conditions in dependence graphs for software safety analysis. *ACM Transactions on Software Engineering and Methodology* 15, 4 (2006), 410–457.
[54] Bjarne Steensgaard. 1996. Points-to analysis in almost linear time. In *Proceedings of the 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 32–41.
[55] Bernhard Steffen, Jens Knoop, and Oliver Rüthing. 1990. The value flow graph: A program representation for optimal program transformations. In *Proceedings of the 3rd European Symposium on Programming Copenhagen (ESOP '90)*. Springer, 389–405.
[56] Benno Stein, Bor-Yuh Evan Chang, and Manu Sridharan. 2021. Demanded abstract interpretation. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 282–295.
[57] Benno Stein, Bor-Yuh Evan Chang, and Manu Sridharan. 2024. Interactive abstract interpretation with demanded summarization. *ACM Transactions on Programming Languages and Systems* 46, 1 (2024), 1–40.
[58] Yulei Sui, Xiao Cheng, Guanqin Zhang, and Haoyu Wang. 2020. Flow2Vec: Value-flow-based precise code embedding. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–27.
[59] Yulei Sui, Peng Di, and Jingling Xue. 2016. Sparse flow-sensitive pointer analysis for multithreaded programs. In *Proceedings of the 2016 International Symposium on Code Generation and Optimization*, 160–170.
[60] Yulei Sui and Jingling Xue. 2016. SVF: Interprocedural static value-flow analysis in LLVM. In *Proceedings of the 25th International Conference on Compiler Construction*, 265–266.
[61] Yulei Sui, Hua Yan, Zheng Zheng, Yunpeng Zhang, and Jingling Xue. 2018. Parallel construction of interprocedural memory SSA form. *Journal of Systems and Software* 146 (2018), 186–195.
[62] Yulei Sui, Ding Ye, and Jingling Xue. 2012. Static memory leak detection using full-sparse value-flow analysis. In *Proceedings of the 2012 International Symposium on Software Testing and Analysis*, 254–264.
[63] Yulei Sui, Ding Ye, and Jingling Xue. 2014. Detecting memory leaks statically with full-sparse value-flow analysis. *IEEE Transactions on Software Engineering* 40, 2 (2014), 107–122.
[64] Tamás Szabó, Gábor Bergmann, Sebastian Erdweg, and Markus Voelter. 2018. Incrementalizing lattice-based program analyses in Datalog. *Proceedings of the ACM on Programming Languages* 2, OOPSLA (2018), 1–29.
[65] Tamás Szabó, Sebastian Erdweg, and Gábor Bergmann. 2021. Incremental whole-program analysis in Datalog with lattices. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 1–15.
[66] Tamás Szabó, Sebastian Erdweg, and Markus Voelter. 2016. IncA: A DSL for the definition of incremental program analyses. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, 320–331.
[67] Robert Tarjan. 1972. Depth-first search and linear graph algorithms. *SIAM Journal on Computing* 1, 2 (1972), 146–160.

[68] Jens Van der Plas, Quentin Stiévenart, Noah Van Es, and Coen De Roover. 2020. Incremental flow analysis through computational dependency reification. In *Proceedings of the 2020 IEEE 20th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 25–36.

[69] Chengpeng Wang, Wenyang Wang, Peisen Yao, Qingkai Shi, Jinguo Zhou, Xiao Xiao, and Charles Zhang. 2023. Anchor: Fast and precise value-flow analysis for containers via memory orientation. *ACM Transactions on Software Engineering and Methodology* 32, 3 (2023), 1–39.

[70] Mark Weiser. 1982. Programmers use slices when debugging. *Communications of the ACM* 25, 7 (1982), 446–452.

[71] Mark Weiser. 1984. Program slicing. *IEEE Transactions on Software Engineering* 4 (1984), 352–357.

[72] John Whaley and Monica S. Lam. 2004. Cloning-based context-sensitive pointer alias analysis using binary decision diagrams. In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation*, 131–144.

[73] Yichen Xie and Alex Aiken. 2005. Context-and path-sensitive memory leak detection. In *Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 115–125.

[74] Xuezheng Xu, Yulei Sui, Hua Yan, and Jingling Xue. 2019. VFix: Value-flow-guided precise program repair for null pointer dereferences. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 512–523.

[75] Hua Yan, Yulei Sui, Shiping Chen, and Jingling Xue. 2018. Spatio-temporal context reduction: A pointer-analysis-based static approach for detecting use-after-free vulnerabilities. In *Proceedings of the 40th International Conference on Software Engineering*, 327–337.

[76] Peisen Yao, Jinguo Zhou, Xiao, Qingkai Shi, Rongxin Wu, and Charles Zhang. 2024. Falcon: A fused approach to path-sensitive sparse data dependence analysis. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 567–592.

[77] Qianshan Yu, Fei He, and Bow-Yaw Wang. 2020. Incremental predicate analysis for regression verification. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–25.

[78] Jyh-shiarn Yur, Barbara G. Ryder, and William A. Landi. 1999. An incremental flow-and context-sensitive pointer aliasing analysis. In *Proceedings of the 21st International Conference on Software Engineering*, 442–451.

[79] Yizhuo Zhai, Yu Hao, Zheng Zhang, Weiteng Chen, Guoren Li, Zhiyun Qian, Chengyu Song, Manu Sridharan, Srikanth V. Krishnamurthy, Trent Jaeger, et al. 2022. Progressive scrutiny: Incremental detection of UBI bugs in the Linux kernel. In *Proceedings of the 2022 Network and Distributed System Security Symposium*.

[80] Sheng Zhan and Jeff Huang. 2016. ECHO: Instantaneous in situ race detection in the IDE. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 775–786.

[81] David Zhao, Pavle Subotic, Mukund Raghothaman, and Bernhard Scholz. 2021. Towards elastic incrementalization for Datalog. In *Proceedings of the 23rd International Symposium on Principles and Practice of Declarative Programming*, 1–16.

[82] Zelin Zhao, Xizao Wang, Zhaogui Xu, Zhenhao Tang, Yongchao Li, and Peng Di. 2023. Incremental call graph construction in industrial practice. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 471–482.

Received 12 August 2024; revised 6 January 2025; accepted 9 March 2025